NATIONAL TECHNICAL UNIVERSITY OF ATHENS

SCHOOL OF APPLIED MATHEMATICS AND PHYSICAL SCIENCES

MSc PROGRAM: MATHEMATICAL MODELING
IN MODERN TECHNOLOGIES AND FINANCIAL ENGINEERING

Bayesian Statistics and MCMC

Final Assignment 2026

Eftychios Kontadakis

e-mail: felkon0@gmail.com

ID: 09325012

Professor:

Dimitrios Fouskakis

Exercise 1

We begin by generating 50 observations for the first 10 of the 15 explanatory variables using a multivariate normal distribution with a zero mean vector and the covariance matrix equal to the identity matrix. This construction effectively produces 10 i.i.d.(independent and identically distributed) variables $X_j \sim N(0,1)$ for $j = 1, \ldots, 10$, each of which has 50 i.i.d. observations.

Thus, for every variable X_j and every observation index i, we have

$$X_{ij} \sim N(0,1), \qquad i = 1, \ldots, 50, \; j = 1, \ldots, 10.$$

Or simply, all these observed values are independent and follow the same normal distribution.

To generate them we will also select a seed so that even though their generation is random, they are always the same every time we run the program, this ensures replicability and allows us to do effective debugging of the code.

```
library(MASS)
set.seed(444) # Seed for replicability and debugging
n <- 50
mu <- rep(0, 10)          # this sets mean to 0 for every X_i
Sigma <- diag(10)         # Covariance matrix = Identity Matrix

X <- mvrnorm(n, mu, Sigma)

# Table of means for testing
data.frame(Variable = paste0("X", 1:ncol(X)),
           Mean = round(colMeans(X), 4))
```

```
   Variable    Mean
1        X1 -0.0842
2        X2  0.1663
3        X3  0.2410
4        X4 -0.0192
5        X5 -0.2460
6        X6  0.3348
7        X7 -0.1677
8        X8  0.1538
9        X9  0.0767
10      X10 -0.2414
```

The mean for every variable is not exactly 0, but that is to be expected. They fluctuate around 0, the more values we generate the closer these means would get to 0.

For the remaining explanatory variables, we simulate values according to the relation:

$$X_{ij} \sim N(0.3X_{i1} + 0.5X_{i2} + 0.7X_{i3} + 0.9X_{i4} + 1.5X_{i5},\ \sigma^2 = 1),$$
$$j = 11, \ldots, 15,\ i = 1, \ldots, 50.$$

These new variables are dependent on $X_1, ..., X_5$, and they follow a different distribution.

```
for (j in 11:15) {
  X <- cbind(X, 0.3*X[,1] + 0.5*X[,2] + 0.7*X[,3] + 0.9*X[,4]
             + 1.5*X[,5] + rnorm(50, 0, 1))
}

# Check means and variances of X11-X15
means_new  <- round(colMeans(X[, 11:15]), 4)
vars_new   <- round(apply(X[, 11:15], 2, var), 4)

data.frame(
  Variable = paste0("X", 11:15),
  Mean     = means_new,
  Variance = vars_new
)
```

```
  Variable    Mean Variance
1      X11 -0.1275   4.1426
2      X12 -0.0983   5.0372
3      X13 -0.2310   4.0648
4      X14 -0.2291   5.5746
5      X15 -0.1473   4.6063
```

The linear combination of the independent $N(0, 1)$ variables is itself normal, and adding an independent $N(0, 1)$ error term preserves normality. Thus, although the noise variance is 1, the total variance of X_{ij} equals the variance of the linear combination plus 1:[1]

$$\sigma^2 = 0.3^2 + 0.5^2 + 0.7^2 + 0.9^2 + 1.5^2 + 1 \approx 4.89$$

giving a final normal distribution with a larger overall spread(≈ 4.89) but the exact same mean (≈ 0)[2].

These values check out with the calculated ones found in the table above.

[1] This is a direct result from $Var(X_1 + X_2) = Var(X_1) + Var(X_2) + 2Cov(X_1, X_2)$ for independent X_1, X_2 the $Cov(X_1, X_2) = 0$ so $Var(X_1 + X_2) = Var(X_1) + Var(X_2)$.

[2] This is a result of $E[X_1 + X_2] = E[X_1] + E[X_2]$, from the linearity of mean, no independency is needed.

We move on to simulating the response variable using **some** of the 15 created variables.

```
# Simulate response variable Y_i
Y <- 4 + 2*X[,1] - X[,5] +
  1.5*X[,7]+ X[,11] + 0.5*X[,13] + rnorm(50, 0, 2.5)

# Check mean and variance of Y_i
mean_Y <- round(mean(Y), 4)
var_Y  <- round(var(Y), 4)

data.frame(Statistic = c("Mean", "Variance"),
           Value = c(mean_Y, var_Y))
```

```
  Statistic   Value
1      Mean  3.6795
2  Variance 16.8131
```

Since Y_i is a linear combination of jointly normal predictors plus an independent $N(0, 2.5^2)$ noise term, it follows that Y_i is itself normally distributed. The mean is simple to calculate since every X_j is normal with mean 0, it follows that $E[Y_i] = 4$, however since these X_j are not independent anymore (X_{11}, X_{13} depend on $X_{1,..,5}$) we cannot calculate the variance by the sum of squares anymore.

By writing Y_i, as an inner product $Y_i = \mathbf{b}^\top \mathbf{X}_i + \varepsilon_i$ of the two vectors $\mathbf{X}_i = (X_{i1}, X_{i5}, X_{i7}, X_{i11}, X_{i13})^\top$ and $\mathbf{b} = (2, -1, 1.5, 1, 0.5)^\top$, with the addition of the independent(to that inner product) ε_i, we can write:

$$\text{Var}(Y_i) = \text{Var}(\mathbf{b}^\top \mathbf{X}_i) + \text{Var}(\varepsilon_i)$$

By expanding $\text{Var}(\mathbf{b}^\top \mathbf{X}_i)$ and recombining the terms we get:

$$\text{Var}(Y_i) = \mathbf{b}^\top \Sigma \mathbf{b} + 2.5^2,$$

where $\Sigma = \Sigma_{ij} = \text{Cov}(\mathbf{X}_i, \mathbf{X}_j)$[3], $\mathbf{X}_i = (X_{i1}, X_{i5}, X_{i7}, X_{i11}, X_{i13})^\top$ and $\mathbf{b} = (2, -1, 1.5, 1, 0.5)^\top$.

> It is important to note that this expression gives a scalar(a singular value) for $Var(Y_i)$, since the product $\mathbf{b}^\top \Sigma \mathbf{b}$ is also a scalar $[1 \times 5] \cdot [5 \times 5] \cdot [5 \times 1] = [1 \times 1]$.

[3] This is a matrix with the variances of each variable in the diagonal and every non-zero covariances on their respective non-diagonal element.

We will now consider the multiple linear regression model

$$Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_{15} X_{15} + \varepsilon \equiv X\beta + \varepsilon, \qquad \varepsilon \sim N(0, \sigma^2),$$

where X is the design matrix of dimension 50×16 and

$$\beta = (\beta_0, \beta_1, \ldots, \beta_{15})^\top.$$

We assume the following conjugate prior distributions:

$$\beta \mid \sigma^2 \sim N_{16}(0,\, \sigma^2 M^{-1}),$$

and,

$$\sigma^2 \sim IG(0.0001,\, 0.0001),$$

The choice of these exact prior for β (and σ^2) is not random; it must depend on σ^2, since the uncertainty induced in the model by ε's variance ultimately determines the uncertainty in β itself. Additionally, because σ^2 follows an Inverse Gamma (IG) distribution and $\beta \mid \sigma^2$ follows a Normal (N) distribution, the joint prior $p(\beta, \sigma^2)$ is a Normal-Inverse Gamma distribution, which when combined with the Normal likelihood originating from Y, the posterior, given by Bayes's Theorem, is also a Normal-Inverse Gamma. Thus it is a conjugate prior, no matter how many times we update with new data, the posterior is always going to be a Normal-Inverse Gamma distribution, with the only difference being the updated parameters. For greater visualization using Bayes Theorem:

$$\underbrace{p(Y \mid \beta, \sigma^2)}_{\text{Normal likelihood}} \times \underbrace{p(\beta, \sigma^2)}_{\text{Normal-IG prior}} \propto \underbrace{p(\beta, \sigma^2 \mid Y)}_{\text{Normal-IG posterior}}$$

Also note that all expressions implicitly condition on the observed design matrix X, which we treat as fixed throughout as it contains already "observed" and thus locked values.

A)

Objective

We are going to prove just that, by showing that the likelihood already resembles a Normal-Inverse Gamma if regarded as a function of the parameters σ^2, β.

Y is a random vector, and by regarding β, σ^2 and X as known quantities, $Y \mid \beta, \sigma^2 \sim N(X\beta, \sigma^2 I_n)$, where I_n is the $n \times n$ identity matrix encoding that all observations share the same variance σ^2 and are independent of each other.

Taking the multivariate Normal density formula:

$$(2\pi)^{-k/2}\det(\Sigma)^{-1/2}\exp\left(-\frac{1}{2}(x-\mu)^\top\Sigma^{-1}(x-\mu)\right)$$

and substituting $k=n$, $\mu=X\beta$, $\Sigma=\sigma^2 I_n$ and $x=y$:

- $\det(\sigma^2 I_n)=(\sigma^2)^n \implies \det(\Sigma)^{-1/2}=(\sigma^2)^{-n/2}$
- $\Sigma^{-1}=(\sigma^2 I_n)^{-1}=\frac{1}{\sigma^2}I_n$[a] $\implies (y-X\beta)^\top\Sigma^{-1}(y-X\beta)=\frac{1}{\sigma^2}(y-X\beta)^\top(y-X\beta)$

and evaluating at the observed realization y gives the likelihood:

$$\ell(\beta,\sigma^2 \mid y)=(2\pi\sigma^2)^{-n/2}\exp\left(-\frac{1}{2\sigma^2}(y-X\beta)^\top(y-X\beta)\right)$$

where $(y-X\beta)^\top(y-X\beta)=\sum_{i=1}^n(y_i-x_i^\top\beta)^2$ is the sum of squared residuals, a scalar, since I_n multiplying any vector leaves it unchanged.

[a] This Σ is a diagonal matrix with σ^2 on the diagonal elements so the inverse Σ^{-1} is again a diagonal matrix but with the inverse diagonal elements.

Our goal is to rewrite the exponent in a more revealing form. To do this we introduce $\hat{\beta}$, the OLS estimator, the value of β that best fits the data with no prior information whatsoever. It is found by minimizing the sum of squared residuals:

$$Q(\beta)=(y-X\beta)^\top(y-X\beta)=\sum_{i=1}^n(y_i-x_i^\top\beta)^2$$

We square the residuals rather than summing them directly because positive and negative errors would otherwise cancel, and squaring penalizes larger errors more heavily while remaining smooth and differentiable everywhere. Expanding:

$$Q(\beta)=y^\top y-\beta^\top X^\top y-y^\top X\beta+\beta^\top X^\top X\beta$$

Taking the derivative with respect to β and setting it to zero:[4]

$$\frac{\partial Q}{\partial\beta}=-2X^\top y+2X^\top X\beta=0 \implies X^\top X\beta=X^\top y$$

[4] Here, for the derivation, we have used the derivative with respect to vectors. For vector derivatives, we have the following identities. If A is an $n\times n$ symmetric matrix, and B another matrix not necessarily symmetric , the following relations are true

$$\frac{\partial}{\partial\mathbf{x}}\left(\tfrac{1}{2}\mathbf{x}^\top A\mathbf{x}\right)=A\mathbf{x},\qquad \frac{\partial}{\partial\mathbf{x}}(B^\top\mathbf{x})=B,\qquad \frac{\partial}{\partial\mathbf{x}}(\mathbf{x}^\top B)=B.$$

These are the normal equations. Solving for β, we inverse $X^\top X$, which can be done since the design matrix X must have full column rank(no perfect multicollinearity), meaning its columns (the predictors) are linearly independent.

$$\hat{\beta} = (X^\top X)^{-1} X^\top y$$

This is the OLS estimator, i.e. the closest β to the data we can find without any prior information, and it is also the Maximum Likelihood Estimator(MLE) under Normal errors (It maximizes the likelihood).

Before introducing the $\hat{\beta}$ to the likelihood, we take the normal equations:

$$X^\top X\hat{\beta} = X^\top y \implies X^\top(y - X\hat{\beta}) = 0 \implies X^\top e = 0$$

where $e = y - X\hat{\beta}$ are the OLS residuals. This says the residuals are perpendicular to every column of X. This is another useful relation we will need for the proof.

> Geometrically, $X\hat{\beta}$ is the closest point to y in the column space of X, and the shortest path from a point to a subspace is always perpendicular to it (setting the derivative to zero and geometric orthogonality are the same statement).

We now rewrite $(y - X\beta)^\top(y - X\beta)$ by adding and subtracting $X\hat{\beta}$ to introduce it:

$$y - X\beta = (y - X\hat{\beta}) + (X\hat{\beta} - X\beta) = e - X(\beta - \hat{\beta})$$

Substituting and expanding using $(a-b)^\top(a-b) = a^\top a - 2a^\top b + b^\top b$[5]:

$$\begin{aligned}&[e - X(\beta - \hat{\beta})]^\top[e - X(\beta - \hat{\beta})]\\ &= e^\top e - 2e^\top X(\beta - \hat{\beta}) + (\beta - \hat{\beta})^\top X^\top X(\beta - \hat{\beta})\end{aligned}$$

The middle term vanishes by orthogonality since $e^\top X = 0$:

$$e^\top X(\beta - \hat{\beta}) = \underbrace{(e^\top X)}_{=0}(\beta - \hat{\beta}) = 0$$

[5] In this relation, we would normally have $(a-b)^\top(a-b) = a^\top a - a^\top b - b^\top a + b^\top b$ but since $a^\top b$, $b^\top a$ are both scalars (plain numbers), the transposed versions are exactly the same or $a^\top b = b^\top a$. By substituting a, b, this won't change as long as the dimensions still match which they do.

We are left with:

$$e^\top e + (\beta - \hat{\beta})^\top X^\top X(\beta - \hat{\beta}) = \underbrace{(y - X\hat{\beta})^\top (y - X\hat{\beta})}_{s^2} + (\beta - \hat{\beta})^\top X^\top X(\beta - \hat{\beta})$$

Conclusion

Substituting back into the likelihood:

$$\ell(\beta, \sigma^2 \mid y) = (2\pi\sigma^2)^{-n/2} \exp\left(-\frac{1}{2\sigma^2}\left[s^2 + (\beta - \hat{\beta})^\top (X^\top X)(\beta - \hat{\beta})\right]\right)$$

where $s^2 = (y - X\hat{\beta})^\top (y - X\hat{\beta})$ is the sum of squared OLS residuals and $\hat{\beta} = (X^\top X)^{-1} X^\top y$ is the MLE.

The split reveals two distinct roles:

- s^2 depends only on the data, not on β.
- $(\beta - \hat{\beta})^\top X^\top X(\beta - \hat{\beta})$ is quadratic in β, centered at $\hat{\beta}$, and gives the likelihood its Normal shape in β

The likelihood can be decomposed into two parts that reveal its Normal-Inverse Gamma shape in (β, σ^2):

$$\ell(\beta, \sigma^2 \mid y) = \underbrace{(2\pi\sigma^2)^{-n/2} \exp\left(-\frac{s^2}{2\sigma^2}\right)}_{\text{Inverse Gamma shape in } \sigma^2} \times \underbrace{\exp\left(-\frac{(\beta - \hat{\beta})^\top (X^\top X)(\beta - \hat{\beta})}{2\sigma^2}\right)}_{\text{Normal shape in } \beta|\sigma^2}$$

Matching against the Normal-Inverse Gamma density, the correspondence is:[6]

- The Normal part has center $\mu \leftrightarrow \hat{\beta}$ and precision $\lambda \leftrightarrow X^\top X$, meaning the likelihood is centered at the OLS estimate $\hat{\beta}$ with uncertainty scaled by σ^2
- The Inverse Gamma part has shape $\alpha \leftrightarrow n/2 - 1$ and rate $\beta \leftrightarrow s^2/2$, where s^2 is the sum of squared OLS residuals

All these confirm that the likelihood, viewed as a function of (β, σ^2), has the shape of an unnormalized Multivariate Normal-Inverse Gamma , justifying the conjugate prior choice made earlier.

[6] The Normal-Inverse Gamma, for reference, is:

$$\frac{\sqrt{\lambda}}{\sqrt{2\pi\sigma^2}} \frac{\beta^\alpha}{\Gamma(\alpha)} \left(\frac{1}{\sigma^2}\right)^{\alpha+1} \exp\left(-\frac{2\beta + \lambda(x - \mu)^2}{2\sigma^2}\right)$$

B)

Posterior Distributions

We are now going to prove that the conditional posterior for β:

$$p(\beta \mid \sigma^2, y, X)$$

is a multivariate Normal distribution of dimension 16:

$$\beta \mid \sigma^2, y, X \sim N_{16}\left((M + X^\top X)^{-1}(X^\top X)\hat{\beta},\ \sigma^2(M + X^\top X)^{-1}\right)$$

and that the marginal posterior for σ^2:

$$p(\sigma^2 \mid y, X)$$

is an Inverse Gamma distribution:

$$\sigma^2 \mid y, X \sim IG\left(\frac{n}{2} + a,\ b + \frac{s^2}{2} + \frac{\hat{\beta}^\top \left(M^{-1} + (X^\top X)^{-1}\right)^{-1} \hat{\beta}}{2}\right)$$

By Bayes theorem the joint posterior is proportional to the likelihood times the prior, and the prior itself can be split in the conditional and marginal[7]:

$$p(\beta, \sigma^2 \mid y) \propto \ell(\beta, \sigma^2 \mid y) \times p(\beta \mid \sigma^2) \times p(\sigma^2)$$

Substituting each piece explicitly:

$$\ell(\beta, \sigma^2 \mid y) \propto (\sigma^2)^{-n/2} \exp\left(-\frac{1}{2\sigma^2}\left[s^2 + (\beta - \hat{\beta})^\top X^\top X(\beta - \hat{\beta})\right]\right)$$

$$p(\beta \mid \sigma^2) \propto (\sigma^2)^{-16/2} \exp\left(-\frac{1}{2\sigma^2}\beta^\top M\beta\right)$$

$$p(\sigma^2) \propto (\sigma^2)^{-(a+1)} \exp\left(-\frac{b}{\sigma^2}\right)$$

[7] Note that,

$$p(\beta, \sigma^2) = p(\beta \mid \sigma^2)\, p(\sigma^2), \text{ by the product rule of probability, } p(x, y) = p(y \mid x)p(x).$$

Multiplying the three pieces together and collecting the powers of σ^2 in front of the exponential:

$$p(\beta, \sigma^2 \mid y) \propto (\sigma^2)^{-n/2-16/2-(a+1)} \exp\left(-\frac{1}{2\sigma^2}\left[s^2 + (\beta-\hat{\beta})^\top X^\top X(\beta-\hat{\beta}) + \beta^\top M\beta + 2b\right]\right)$$

We now focus on the terms inside the exponential that involve β:

$$(\beta-\hat{\beta})^\top X^\top X(\beta-\hat{\beta}) + \beta^\top M\beta$$

Expanding $(\beta-\hat{\beta})^\top X^\top X(\beta-\hat{\beta})$:

$$= \beta^\top X^\top X\beta - 2\hat{\beta}^\top X^\top X\beta + \hat{\beta}^\top X^\top X\hat{\beta}$$

So the full β dependent part is:

$$\beta^\top X^\top X\beta - 2\hat{\beta}^\top X^\top X\beta + \hat{\beta}^\top X^\top X\hat{\beta} + \beta^\top M\beta$$

$$= \beta^\top (M + X^\top X)\beta - 2\hat{\beta}^\top (X^\top X)\beta + \hat{\beta}^\top X^\top X\hat{\beta}$$

Hint

We now apply the identity provided in the assignment:

$$\beta^\top (M + X^\top X)\beta - 2\hat{\beta}^\top (X^\top X)\beta =$$

$$= \left(\beta - (M + X^\top X)^{-1}(X^\top X)\hat{\beta}\right)^\top (M + X^\top X)\left(\beta - (M + X^\top X)^{-1}(X^\top X)\hat{\beta}\right)$$

$$-\hat{\beta}^\top (X^\top X)(M + X^\top X)^{-1}(X^\top X)\hat{\beta}$$

Denoting $\tilde{\beta} = (M + X^\top X)^{-1}(X^\top X)\hat{\beta}$ for brevity, the exponent becomes:

$$s^2 + (\beta-\tilde{\beta})^\top (M + X^\top X)(\beta-\tilde{\beta}) + \hat{\beta}^\top X^\top X\hat{\beta} - \hat{\beta}^\top (X^\top X)(M + X^\top X)^{-1}(X^\top X)\hat{\beta} + 2b$$

Hint

We will now apply the second identity provided in the assignment:

$$(M + X^\top X)^{-1} = (X^\top X)^{-1} - (X^\top X)^{-1} \left(M^{-1} + (X^\top X)^{-1}\right)^{-1} (X^\top X)^{-1}$$

With that, we can simplify $\hat{\beta}^\top X^\top X\hat{\beta} - \hat{\beta}^\top (X^\top X)(M + X^\top X)^{-1}(X^\top X)\hat{\beta}$ by substituting into the second term:

$$\begin{aligned}
&\hat{\beta}^\top (X^\top X)(M + X^\top X)^{-1}(X^\top X)\hat{\beta} \\
&= \hat{\beta}^\top (X^\top X) \left[(X^\top X)^{-1} - (X^\top X)^{-1} \left(M^{-1} + (X^\top X)^{-1}\right)^{-1} (X^\top X)^{-1}\right] (X^\top X)\hat{\beta} \\
&= \hat{\beta}^\top X^\top X\hat{\beta} - \hat{\beta}^\top \left(M^{-1} + (X^\top X)^{-1}\right)^{-1} \hat{\beta}
\end{aligned}$$

Subtracting from $\hat{\beta}^\top X^\top X\hat{\beta}$:

$$\begin{aligned}
&\hat{\beta}^\top X^\top X\hat{\beta} - \hat{\beta}^\top (X^\top X)(M + X^\top X)^{-1}(X^\top X)\hat{\beta} \\
&= \hat{\beta}^\top X^\top X\hat{\beta} - \hat{\beta}^\top X^\top X\hat{\beta} + \hat{\beta}^\top \left(M^{-1} + (X^\top X)^{-1}\right)^{-1} \hat{\beta} \\
&= \hat{\beta}^\top \left(M^{-1} + (X^\top X)^{-1}\right)^{-1} \hat{\beta}
\end{aligned}$$

So the full exponent simplifies to:

$$s^2 + (\beta - \tilde{\beta})^\top (M + X^\top X)(\beta - \tilde{\beta}) + \hat{\beta}^\top \left(M^{-1} + (X^\top X)^{-1}\right)^{-1} \hat{\beta} + 2b$$

The full joint posterior is therefore:

$$\begin{aligned}
&p(\beta, \sigma^2 \mid y) \propto (\sigma^2)^{-n/2-16/2-(a+1)} \\
&\times \exp\left(-\frac{1}{2\sigma^2}\left[(\beta - \tilde{\beta})^\top (M + X^\top X)(\beta - \tilde{\beta}) + s^2 + \hat{\beta}^\top \left(M^{-1} + (X^\top X)^{-1}\right)^{-1} \hat{\beta} + 2b\right]\right)
\end{aligned}$$

This splits naturally into two parts:

$$\begin{aligned}
&= \underbrace{\exp\left(-\frac{1}{2\sigma^2}(\beta - \tilde{\beta})^\top (M + X^\top X)(\beta - \tilde{\beta})\right)}_{\text{Normal shape in } \beta|\sigma^2} \\
&\underbrace{\times (\sigma^2)^{-n/2-16/2-(a+1)} \exp\left(-\frac{1}{\sigma^2}\left[b + \frac{s^2}{2} + \frac{\hat{\beta}^\top \left(M^{-1} + (X^\top X)^{-1}\right)^{-1} \hat{\beta}}{2}\right]\right)}_{\text{Inverse Gamma shape in } \sigma^2}
\end{aligned}$$

Conclusion

Conditional posterior of $\beta \mid \sigma^2, y$:
Fixing σ^2 and looking at the joint posterior as a function of β alone, only the quadratic term remains:

$$p(\beta \mid \sigma^2, y) \propto \exp\left(-\frac{1}{2\sigma^2}(\beta - \tilde{\beta})^\top (M + X^\top X)(\beta - \tilde{\beta})\right)$$

This is the kernel of a multivariate Normal distribution, giving:

$$\boxed{\beta \mid \sigma^2, y \sim N_{16}\left(\tilde{\beta},\ \sigma^2 (M + X^\top X)^{-1}\right)}$$

where $\tilde{\beta} = (M + X^\top X)^{-1}(X^\top X)\hat{\beta}$.
Marginal posterior of $\sigma^2 \mid y$:
Integrating out β, the Gaussian integral over β contributes only a constant with respect to σ^2, leaving:

$$p(\sigma^2 \mid y) \propto (\sigma^2)^{-n/2-16/2-(a+1)} \exp\left(-\frac{1}{\sigma^2}\left[b + \frac{s^2}{2} + \frac{\hat{\beta}^\top \left(M^{-1} + (X^\top X)^{-1}\right)^{-1} \hat{\beta}}{2}\right]\right)$$

This is the kernel of an Inverse Gamma distribution, giving:

$$\boxed{\sigma^2 \mid y \sim IG\left(\frac{n}{2} + a,\ b + \frac{s^2}{2} + \frac{\hat{\beta}^\top \left(M^{-1} + (X^\top X)^{-1}\right)^{-1} \hat{\beta}}{2}\right)}$$

confirming that the joint posterior is Normal-Inverse Gamma with updated parameters incorporating both the data through s^2, $\hat{\beta}$ and the prior through a, b, M.

C)

MCMC

Using JAGS (Just Another Gibbs Sampler), we are going to simulate values from the posterior distributions of β_j for $j = 0, \ldots, 15$ and σ^2 assuming that M is a diagonal matrix where all diagonal values are equal to 10^{-4}. We are then going to compare our results with those from classical statistics with Ordinary Least Squares (OLS) and compare the posterior distributions of the regression coefficients using boxplots. We are also going to determine which explanatory variables should be removed from the model based on those plots, and check if the true values of the coefficients lie at the center of the posterior distributions.

Note: Since WinBUGS uses identical model syntax to JAGS and both implement the same MCMC algorithms, we use JAGS via the `R2jags` package which is more stable on modern systems. The results are equivalent.

The JAGS model is specified with the following components:

- **Likelihood:** Each observation $Y_i \sim N(\mu_i, \tau)$ where $\tau = 1/\sigma^2$ is the precision and $\mu_i = \beta_0 + \sum_{j=1}^{15} \beta_j X_{ij}$ is the linear predictor.[8]
- **Prior on β:** Each $\beta_j \sim N(0, \tau/10000)$, corresponding to $\beta \mid \sigma^2 \sim N_{16}(0, \sigma^2 M^{-1})$ with $M^{-1} = 10000 \cdot I_{16}$ since $M = 10^{-4} \cdot I_{16}$, i.e. prior variance $\sigma^2 \cdot 10000$ per coefficient. In JAGS precision notation, this is $\tau/10000$. Since the prior variance is $10000\sigma^2$, which is very large relative to the data scale, this is a genuinely **vague prior** that allows the data to dominate the posterior.
- **Prior on τ:** $\tau \sim \text{Gamma}(0.0001, 0.0001)$, which corresponds to $\sigma^2 \sim IG(0.0001, 0.0001)$ — a nearly flat, non-informative prior on the noise variance.

We run 3 independent chains for 10000 iterations each, discarding the first 2000 as burn-in, giving $3 \times 8000 = 24000$ posterior samples per parameter. Multiple chains allow us to compute the Gelman-Rubin diagnostic $\hat{R}$ to verify convergence.

```
suppressPackageStartupMessages({
library(MASS)
library(R2jags)})
# We regenerate the data with the same seed 444 to ensure
# identical observations across all chunks
set.seed(444)
n <- 50
mu_vec <- rep(0, 10)
Sigma  <- diag(10)
X_mat  <- mvrnorm(n, mu_vec, Sigma)

for (j in 11:15) {
  X_mat <- cbind(
    X_mat,
    0.3*X_mat[,1] + 0.5*X_mat[,2] +
    0.7*X_mat[,3] + 0.9*X_mat[,4] +
    1.5*X_mat[,5] + rnorm(50, 0, 1)
  )}
Y <- 4 + 2*X_mat[,1] - X_mat[,5] + 1.5*X_mat[,7] +
     X_mat[,11] + 0.5*X_mat[,13] + rnorm(50, 0, 2.5)

# JAGS requires all data passed explicitly as a named list
```

[8] Note that JAGS parametrizes the Normal distribution by precision rather than variance.

```r
# since it cannot access R's environment directly
jags_data <- list(y = Y, X = X_mat, n = n)

# Starting values, neutral, burn-in ensures
# they do not affect the final posterior
jags_inits <- function() list(beta = rep(0, 16), tau = 1)

# Parameters to monitor
jags_params <- c("beta", "sigma2")

# Model written as a string to be saved to a file
# and passed to the JAGS executable
jags_model <- "
model {
  for (i in 1:n) {
    y[i]  ~ dnorm(mu[i], tau)
    mu[i] <- beta[1] + inprod(beta[2:16], X[i,])
  }
  for (j in 1:16) {
    beta[j] ~ dnorm(0, tau/10000)
  }
  tau    ~ dgamma(0.0001, 0.0001)
  sigma2 <- 1 / tau
}
"
model_file <- file.path(tempdir(), "model_jags.txt")
writeLines(jags_model, model_file)

fit <- jags(
  data               = jags_data,
  inits              = jags_inits,
  parameters.to.save = jags_params,
  model.file         = model_file,
  n.chains           = 3,
  n.iter             = 10000,
  n.burnin           = 2000,
  n.thin             = 1
)
```

```r
library(knitr)
summary_df <- as.data.frame(round(fit$BUGSoutput$summary, 4))
rownames(summary_df) <- c(paste0("beta[", 0:15, "]"), "deviance",
                          "sigma2")
summary_df <- summary_df[rownames(summary_df) != "deviance", ]
summary_df <- summary_df[, !colnames(summary_df) %in% c("n.eff",
                          "Rhat")]
kable(summary_df, caption = "Posterior summary for all parameters")
```

Table 1: Posterior summary for all parameters

	mean	sd	2.5%	25%	50%	75%	97.5%
beta[0]	3.5189	0.3582	2.8243	3.2813	3.5168	3.7573	4.2243
beta[1]	1.4929	0.4122	0.6820	1.2182	1.4913	1.7696	2.3021
beta[2]	-0.0058	0.5017	-0.9964	-0.3413	-0.0058	0.3228	0.9852
beta[3]	0.1159	0.5861	-1.0299	-0.2831	0.1209	0.5076	1.2508
beta[4]	-0.6119	0.6907	-1.9695	-1.0752	-0.6146	-0.1503	0.7568
beta[5]	-1.9633	1.0995	-4.1054	-2.7032	-1.9621	-1.2353	0.2267
beta[6]	0.8764	0.3866	0.1118	0.6167	0.8794	1.1342	1.6348
beta[7]	1.7833	0.2837	1.2234	1.5946	1.7833	1.9732	2.3379
beta[8]	0.7224	0.3408	0.0465	0.4940	0.7246	0.9496	1.3915
beta[9]	-0.6613	0.3029	-1.2626	-0.8619	-0.6594	-0.4570	-0.0647
beta[10]	0.1270	0.3538	-0.5732	-0.1075	0.1287	0.3597	0.8244
beta[11]	1.4006	0.3047	0.7955	1.2001	1.4026	1.6043	1.9971
beta[12]	0.1510	0.3483	-0.5354	-0.0804	0.1500	0.3839	0.8359
beta[13]	-0.1100	0.2772	-0.6541	-0.2948	-0.1085	0.0747	0.4343
beta[14]	-0.0968	0.2578	-0.6006	-0.2688	-0.0985	0.0758	0.4138
beta[15]	0.7555	0.3114	0.1455	0.5480	0.7562	0.9630	1.3639
sigma2	4.0365	0.8516	2.7064	3.4302	3.9171	4.5121	6.0333

Convergence Diagnostics:

Before interpreting results we assess convergence through trace plots, the Gelman-Rubin diagnostic $\hat{R}$, effective sample sizes, and autocorrelation plots.

We select a representative subset of parameters for diagnostic plots, namely one coefficient with true non-zero values ($\beta_1 = 2$) , one with a coefficient that is zero ($\beta_2 = 0$) and σ^2 which is always important to monitor independently.

```
library(coda)
mcmc_samples <- as.mcmc(fit)
# Rename by matching, not by position
for (i in 1:length(mcmc_samples)) {
  cn <- colnames(mcmc_samples[[i]])
  for (j in 1:16) {
    cn[cn == paste0("beta[", j, "]")] <- paste0("beta[", j-1, "]")
  }
  colnames(mcmc_samples[[i]]) <- cn}

key_params <- c("beta[1]",    # true = 2
                "beta[2]",    # true = 0
                "sigma2")     # true = 6.25

param_labels <- c("beta[1] (true value = 2)",
                  "beta[2] (true value = 0)",
                  "sigma2  (true value = 6.25)")
```

```r
par(mfrow = c(3, 2), mar = c(4, 4, 3, 1))
for (i in seq_along(key_params)) {
  plot(mcmc_samples[, key_params[i]],
       main = param_labels[i],
       auto.layout = FALSE)}
```

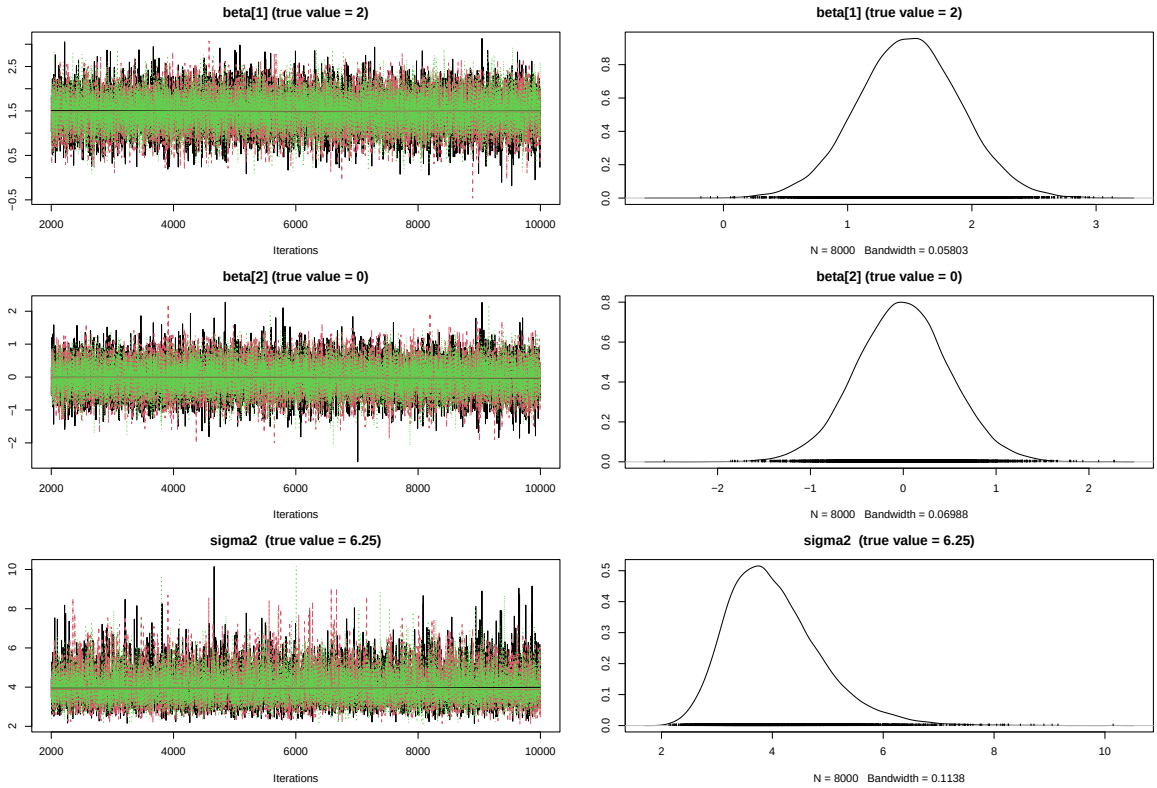

Figure 1: Trace plots (left) and posterior kernel density estimates (right) for three representative parameters. Row 1: β_1 (true value 2), a truly non-zero coefficient. Row 2: β_2 (true value 0), a truly zero coefficient. Row 3: σ^2, the variance.

Trace plots (left column): All three parameters show the characteristic stationary, well-mixed "hairy caterpillar'' pattern across all 3 chains (black, red, green). No trending, drifting or getting stuck is observed, confirming that the burn-in of 2000 iterations was sufficient and the chains have converged to the stationary posterior distribution.

Density plots (right column): The three parameters show distinct posterior behaviors:

- β_1 (true value 2) — posterior centered around $\approx$ 1.5, correctly identifying a non-zero positive effect. The slight underestimation of the true value is attributable to multicollinearity with the dependent variables X_{11}, X_{13}

- β_2 (true value 0) — posterior tightly centered at zero with mass concentrated in $[-1, 1]$, correctly identifying this coefficient as negligible
- σ^2 posterior centered around ≈ 4 with a right skew characteristic of the Inverse Gamma distribution. Here, σ^2 appears to be underestimated compared with the true simulation value $2.5^2 = 6.25$. This is not necessarily problematic, as it may be explained by the relatively small sample size, the large number of predictors, and the presence of strong multicollinearity among some explanatory variables. These factors can allow the fitted model to absorb part of the random variation, leading to smaller residuals and therefore a lower estimate of the residual variance.

We also do autocorrelation plots, to see whether the next value in the chain is dependent on the previous ones.

```
par(mfrow = c(3, 1), mar = c(4, 4, 3, 1))

for (i in seq_along(key_params)) {
  autocorr.plot(mcmc_samples[[1]][, key_params[i]],
                main = param_labels[i],
                auto.layout = FALSE)
}
```

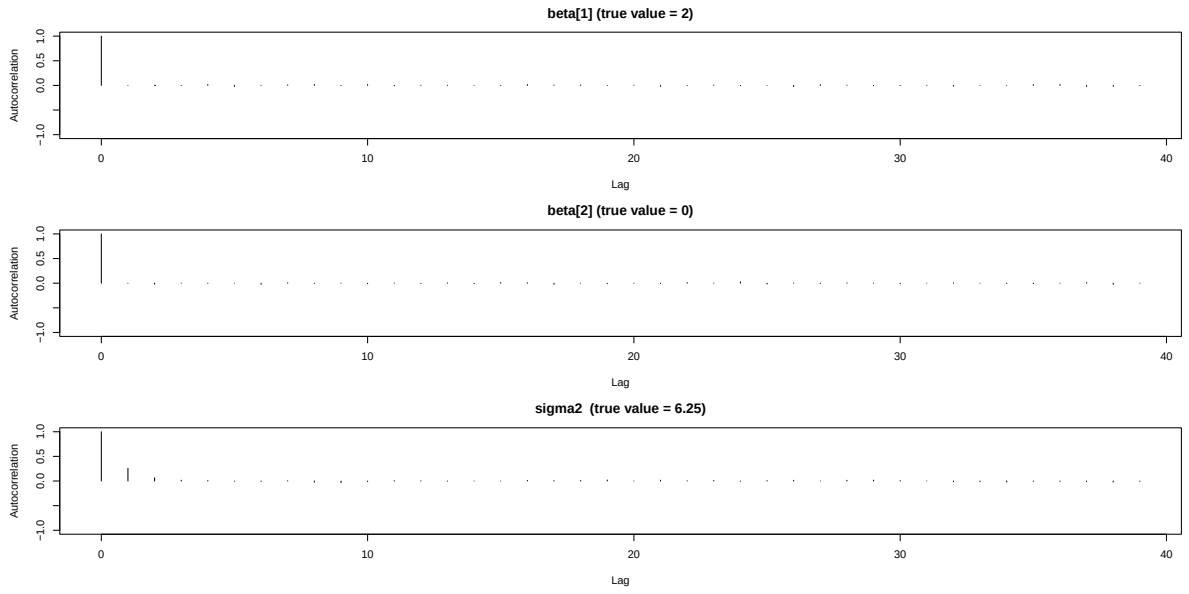

Figure 2: Autocorrelation plots for three representative parameters from chain 1. Lag 0 always equals 1 by definition. Quick decay to zero indicates low autocorrelation between consecutive samples.

Autocorrelation drops to near zero immediately after lag 1 for all parameters, confirming that consecutive samples are essentially independent. This means the nominal 24000 samples are close to 24000 effectively independent draws.

We are now going to use the Gelman-Rubin Statistic ($\hat{R}$)[9] to check whether the 3 MCMC chains have converged to the same posterior distribution.

```
gelman <- gelman.diag(mcmc_samples)
gelman_df <- data.frame(
  Parameter = rownames(gelman$psrf),
  Point_Est = round(gelman$psrf[, 1], 4),
  Upper_CI  = round(gelman$psrf[, 2], 4),
  row.names = NULL
)
# Sort
correct_order <- c(paste0("beta[", 0:15, "]"), "sigma2")
gelman_df <- gelman_df[match(correct_order, gelman_df$Parameter), ]
kable(gelman_df, caption = "Gelman-Rubin convergence diagnostic")
```

Table 2: Gelman-Rubin convergence diagnostic

	Parameter	Point_Est	Upper_CI
1	beta[0]	1.0001	1.0005
9	beta[1]	1.0001	1.0005
10	beta[2]	1.0001	1.0004
11	beta[3]	1.0003	1.0006
12	beta[4]	1.0001	1.0001
13	beta[5]	1.0001	1.0002
14	beta[6]	1.0000	1.0001
15	beta[7]	1.0000	1.0002
16	beta[8]	1.0004	1.0016
2	beta[9]	1.0003	1.0013
3	beta[10]	1.0000	1.0002
4	beta[11]	1.0000	1.0000
5	beta[12]	1.0000	1.0000
6	beta[13]	0.9999	1.0000
7	beta[14]	1.0001	1.0005
8	beta[15]	1.0001	1.0002
18	sigma2	1.0001	1.0004

All $\hat{R}$ values are extremely close to 1 (maximum observed ≈ 1.002), well below the standard threshold of 1.1. This confirms that all 3 chains have converged to the same stationary distribution.

We are now going to calculate the Effective Sample Size(ESS)[10], to check how many independent draws the MCMC chains out of the 24000.

[9]The Gelman–Rubin statistic $\hat{R}$ compares the variance within MCMC chains to the variance between chains: $\hat{R} = \sqrt{\frac{\hat{V}}{W}}$, where $\hat{V} = \frac{n-1}{n}W + \frac{1}{n}B.$, W is the average within-chain variance, B is the between-chain variance.
If chains have converged, $B \approx W$ and $\hat{R} \approx 1.$, indicating convergence.

[10]The effective sample size (ESS) measures how many independent draws an MCMC chain is equivalent to. The effective sample

```r
eff_size <- effectiveSize(mcmc_samples)
eff_df <- data.frame(
  Parameter = names(eff_size),
  ESS       = round(as.numeric(eff_size), 0),
  row.names = NULL)
# Sort
eff_df <- eff_df[match(correct_order, eff_df$Parameter), ]
kable(eff_df, caption = "Effective sample sizes")
```

Table 3: Effective sample sizes

	Parameter	ESS
1	beta[0]	24919
9	beta[1]	24000
10	beta[2]	24000
11	beta[3]	24722
12	beta[4]	24526
13	beta[5]	24000
14	beta[6]	23540
15	beta[7]	23618
16	beta[8]	24815
2	beta[9]	23923
3	beta[10]	24339
4	beta[11]	24645
5	beta[12]	24000
6	beta[13]	25020
7	beta[14]	24939
8	beta[15]	24000
18	sigma2	14432

All parameters achieve an ESS close to the nominal 24000, confirming negligible autocorrelation. The exception is σ^2 with ESS ≈ 14000, which is still well above the 1000 threshold for reliable inference[11].

> Note that some ESS exceed 24000, this means that they actually show a small negative autocorrelation.

size is ESS $= \frac{mn}{1+2\sum_{k=1}^{\infty}\rho_k}$,, where m is the number of chains, n the samples per chain, and ρ_k the lag-k autocorrelation. Higher autocorrelation lowers ESS. It is essentially an equivalent to the MC error.

[11] This 1000 sample threshold is more conservative than the usual < 0.05 we use in the MC error, which translates to only 400 samples being independent.

Classical OLS Comparison:

```
ols_fit   <- lm(Y ~ X_mat)
true_vals <- c(4, 2, 0, 0, 0, -1, 0, 1.5, 0, 0, 0, 1, 0, 0.5, 0, 0)

bayes_summary <- fit$BUGSoutput$summary[1:16, ]

# Extract OLS standard errors from summary
ols_se <- summary(ols_fit)$coefficients[, "Std. Error"]

comparison <- data.frame(
  Parameter    = paste0("beta[", 0:15, "]"),
  True_Value   = true_vals,
  OLS_Estimate = round(coef(ols_fit), 4),
  OLS_SD       = round(ols_se, 4),
  Bayes_Mean   = round(bayes_summary[, "mean"],  4),
  Bayes_SD     = round(bayes_summary[, "sd"],    4),
  CI_2.5       = round(bayes_summary[, "2.5%"],  4),
  CI_97.5      = round(bayes_summary[, "97.5%"], 4),
  row.names    = NULL
)

kable(comparison,
      caption = "Comparison of true values, OLS estimates and
      Bayesian posterior summaries")
```

Table 4: Comparison of true values, OLS estimates and Bayesian posterior summaries

Parameter	True_Value	OLS_Estimate	OLS_SD	Bayes_Mean	Bayes_SD	CI_2.5	CI_97.5
beta[0]	4.0	3.5151	0.4249	3.5189	0.3582	2.8243	4.2243
beta[1]	2.0	1.4913	0.4911	1.4929	0.4122	0.6820	2.3021
beta[2]	0.0	-0.0060	0.5959	-0.0058	0.5017	-0.9964	0.9852
beta[3]	0.0	0.1124	0.6993	0.1159	0.5861	-1.0299	1.2508
beta[4]	0.0	-0.6202	0.8205	-0.6119	0.6907	-1.9695	0.7568
beta[5]	-1.0	-1.9730	1.3105	-1.9633	1.0995	-4.1054	0.2267
beta[6]	0.0	0.8829	0.4595	0.8764	0.3866	0.1118	1.6348
beta[7]	1.5	1.7844	0.3362	1.7833	0.2837	1.2234	2.3379
beta[8]	0.0	0.7209	0.4018	0.7224	0.3408	0.0465	1.3915
beta[9]	0.0	-0.6623	0.3599	-0.6613	0.3029	-1.2626	-0.0647
beta[10]	0.0	0.1342	0.4195	0.1270	0.3538	-0.5732	0.8244
beta[11]	1.0	1.4042	0.3607	1.4006	0.3047	0.7955	1.9971
beta[12]	0.0	0.1570	0.4161	0.1510	0.3483	-0.5354	0.8359
beta[13]	0.5	-0.1116	0.3309	-0.1100	0.2772	-0.6541	0.4343
beta[14]	0.0	-0.0970	0.3054	-0.0968	0.2578	-0.6006	0.4138
beta[15]	0.0	0.7550	0.3694	0.7555	0.3114	0.1455	1.3639

The Bayesian posterior means are virtually identical to the OLS estimates throughout, and the posterior standard deviations are consistently slightly smaller than the OLS standard errors. Both observations are expected, with prior variance $10000\sigma^2$ per coefficient the prior is so vague that the likelihood dominates completely and Bayesian inference reduces to classical inference. The marginal reduction in posterior standard deviations reflects the small but non-zero contribution of the prior, which pulls estimates very gently toward zero, or to put it simply, the prior adds a tiny bit of extra information, reducing uncertainty very slightly.

Posterior Boxplots:

We will now do boxplots for all posterior parameters in order to visualize where their estimated values lie. By examining the center and spread of each box, we can see whether the bulk of the posterior mass is close to zero or clearly shifted away from it. Parameters whose credible intervals include 0 are potential candidates for removal, since they show weak evidence of contributing to the model. In contrast, parameters whose boxplots lie clearly away from zero indicate stronger, more meaningful effects.

> It is important to note that while the boxplots provide a useful visual summary of where each posterior coefficient lies, they are not sufficient on their own for deciding which parameters to remove. However, having already examined multiple diagnostics (the Gelman–Rubin statistic, the effective sample size, and the autocorrelation structure), we ensure that the posterior estimates are stable, well-mixed, and based on effectively independent samples. Under these conditions, coefficients whose credible intervals include zero can be considered reliable candidates for removal.

```
beta_samples <- fit$BUGSoutput$sims.matrix[, 1:16]
colnames(beta_samples) <- paste0("β[", 0:15, "]")
par(mar = c(5, 4, 4, 2))
boxplot(beta_samples,
        main   = "Posterior Distributions of Regression Coefficients",
        ylab   = "Posterior Value",
        xlab   = "Coefficient",
        las    = 2,
        col    = "lightblue",
        border = "darkblue")
abline(h = 0, col = "red", lty = 2, lwd = 2)
points(1:16, true_vals, pch = 18, col = "orange", cex = 1.5)
legend("topright",
       legend = c("Zero reference", "True value"),
       col    = c("red", "orange"),
       lty    = c(2, NA),
       pch    = c(NA, 18),
       lwd    = c(2, NA))
```

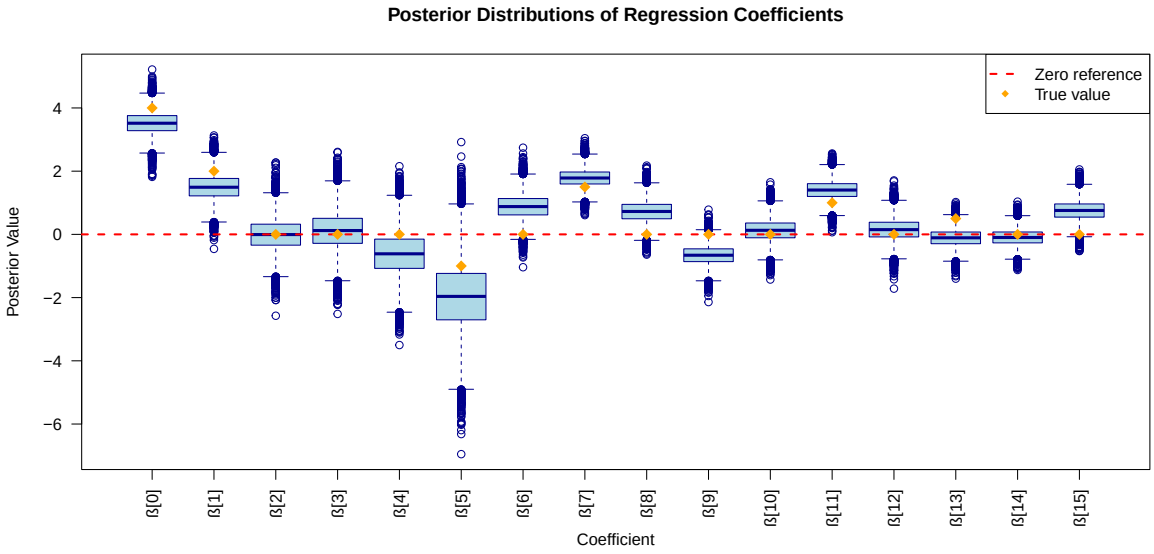

Figure 3: The boxplots show the full posterior distribution of each β_j. The red dashed line marks zero and the orange diamonds mark the true simulated values. The boxes themselves are the IQR, which contains the middle 50% of the posterior samples (25th to 75th percentile), while the whiskers go to the most extreme points within 1.5xIQR from the box.

From the boxplots and credible intervals:

- $\beta_0, \beta_1, \beta_7, \beta_{11}$'s credible intervals clearly exclude zero, correctly identified them as important
- β_5 is a borderline case, the CI barely includes zero ($[-4.11, 0.23]$) due to multicollinearity: $X_{11}, \ldots, X_{15}$ depend on X_5, causing the model to partially absorb β_5's signal through the correlated variables
- β_{13} (true value 0.5) is missed entirely, the effect is too small to detect reliably with only $n = 50$ observations
- β_{15} is a false positive due to multicollinearity, as X_{15} is one of the dependent variables built from $X_1, \ldots, X_5$, sharing signal with the true predictors through its construction. β_6, β_8, and β_9 are false positives due to sampling variability($n = 50$ observations and $p = 15$ predictors)
- All remaining coefficients, namely $\beta_2, \beta_3, \beta_4, \beta_{10}, \beta_{12}, \beta_{14}$, are correctly identified as near zero and are candidates for removal

With $n = 50$ and $p = 15$, the model has limited degrees of freedom, making it difficult to reliably distinguish true signals from noise, whether through multicollinearity among the correlated predictors $X_{11}, \ldots, X_{15}$ or through chance correlations in a small sample.

Finally, to check if the real values are in the center of the posteriors.

```
coverage <- data.frame(
  Parameter  = paste0("beta[", 0:15, "]"),
  True_Value = true_vals,
  CI_2.5     = round(bayes_summary[, "2.5%"],  4),
  CI_97.5    = round(bayes_summary[, "97.5%"], 4),
  Covered    = ifelse(
    true_vals >= bayes_summary[, "2.5%"] &
    true_vals <= bayes_summary[, "97.5%"],
    "Yes", "No"),
  row.names  = NULL)
kable(coverage, caption = "Coverage of 95% Credible Intervals
      for Each Parameter")
```

Table 5: Coverage of 95% Credible Intervals for Each Parameter

Parameter	True_Value	CI_2.5	CI_97.5	Covered
beta[0]	4.0	2.8243	4.2243	Yes
beta[1]	2.0	0.6820	2.3021	Yes
beta[2]	0.0	-0.9964	0.9852	Yes
beta[3]	0.0	-1.0299	1.2508	Yes
beta[4]	0.0	-1.9695	0.7568	Yes
beta[5]	-1.0	-4.1054	0.2267	Yes
beta[6]	0.0	0.1118	1.6348	No
beta[7]	1.5	1.2234	2.3379	Yes
beta[8]	0.0	0.0465	1.3915	No
beta[9]	0.0	-1.2626	-0.0647	No
beta[10]	0.0	-0.5732	0.8244	Yes
beta[11]	1.0	0.7955	1.9971	Yes
beta[12]	0.0	-0.5354	0.8359	Yes
beta[13]	0.5	-0.6541	0.4343	No
beta[14]	0.0	-0.6006	0.4138	Yes
beta[15]	0.0	0.1455	1.3639	No

It is important to note that the credible intervals shown here are the true 95% posterior credible intervals, computed from the 2.5% and 97.5% posterior quantiles. In contrast, the boxplots use the classical definition of whiskers, extending to

$$1.5 \times \text{IQR},$$

which is not the same as the 95% credible interval. As a result, the limits in the boxplots and the limits in the table do not coincide, and they should not be interpreted as representing the same quantity.

```
proximity <- data.frame(
  Parameter  = paste0("beta[", 0:15, "]"),
  True_Value = true_vals,
  Post_Mean  = round(bayes_summary[, "mean"], 4),
  Post_SD    = round(bayes_summary[, "sd"],   4),
  Dist_SDs   = round(abs(true_vals - bayes_summary[, "mean"]) /
                     bayes_summary[, "sd"], 2),
  At_Center  = ifelse(
  abs(true_vals - bayes_summary[, "mean"])
  / bayes_summary[, "sd"] < 1, "Yes", "No"),
  row.names  = NULL)
kable(proximity,
col.names = c("Parameter", "True", "Post. Mean", "Post. SD",
                    "|true - mean| / SD", "Within 1 SD?"),
caption   = "Proximity of true coefficient values
to posterior means")
```

Table 6: Proximity of true coefficient values to posterior means

Parameter	True	Post. Mean	Post. SD	|true - mean| / SD	Within 1 SD?
beta[0]	4.0	3.5189	0.3582	1.34	No
beta[1]	2.0	1.4929	0.4122	1.23	No
beta[2]	0.0	-0.0058	0.5017	0.01	Yes
beta[3]	0.0	0.1159	0.5861	0.20	Yes
beta[4]	0.0	-0.6119	0.6907	0.89	Yes
beta[5]	-1.0	-1.9633	1.0995	0.88	Yes
beta[6]	0.0	0.8764	0.3866	2.27	No
beta[7]	1.5	1.7833	0.2837	1.00	Yes
beta[8]	0.0	0.7224	0.3408	2.12	No
beta[9]	0.0	-0.6613	0.3029	2.18	No
beta[10]	0.0	0.1270	0.3538	0.36	Yes
beta[11]	1.0	1.4006	0.3047	1.31	No
beta[12]	0.0	0.1510	0.3483	0.43	Yes
beta[13]	0.5	-0.1100	0.2772	2.20	No
beta[14]	0.0	-0.0968	0.2578	0.38	Yes
beta[15]	0.0	0.7555	0.3114	2.43	No

Conclusion

Of the 16 parameters, 11 have their true value covered by the 95% credible interval. The 5 failures are not surprising:

- $\beta_6, \beta_8, \beta_9, \beta_{15}$ — true value is 0 but the CI excludes zero due to multicollinearity absorbing signal from correlated predictors (false positives)
- β_{13}, true value is 0.5 but the CI includes only values near zero; the effect is too weak to detect at $n = 50$ (false negative)

The proximity table above quantifies this directly using the standardised distance $|\text{true} - \text{mean}|/\text{SD}$. Only β_7 (distance $= 1.00$) sits essentially at the center of its posterior. The other signal-carrying coefficients, namely β_0 (1.34), β_1 (1.23), and β_{11} (1.31), are within roughly 1.3 SDs, meaning the true value falls in the denser part of the posterior but is not at its peak. This mild displacement is a direct consequence of multicollinearity: the dependent variables X_{11}, X_{13} share signal with X_1, causing the posterior for β_1 (and similarly β_{11}) to shift away from the true value. For the truly-zero coefficients that are correctly identified ($\beta_2, \beta_3, \beta_4, \beta_{10}, \beta_{12}, \beta_{14}$), the distance is negligible (≤ 0.43) since both the true value and the posterior mean are near zero.

The false positive parameters (β_6, β_8, β_9, β_{15}) and β_{13} show the largest displacements (> 2 SDs from their true values). For β_{15} this reflects multicollinearity pulling the posterior away from zero. For β_6, β_8, and β_9 it reflects spurious correlations with Y in this particular sample of $n = 50$. For β_{13} the effect size of 0.5 is simply too small to detect reliably, with the posterior mean near zero rather than near the true value.

Convergence diagnostics were excellent: trace plots showed stable mixing, autocorrelation dropped immediately, effective sample sizes were large, and all Gelman–Rubin statistics satisfied $\hat{R} \approx 1.000$. Overall, the MCMC results confirmed the Normal–Inverse-Gamma posterior structure derived analytically and agreed closely with the OLS estimates for the significant predictors.

D)

Custom Gibbs Sampler

We are now going to implement our own Gibbs sampler in R in order to directly simulate from the posterior distributions of the regression coefficients $\beta_0, \ldots, \beta_{15}$ and the noise variance σ^2. This allows us to verify in practice the Normal–Inverse-Gamma posterior structure derived analytically in parts A and B, and to compare the results of our custom sampler with both the classical OLS estimates and the JAGS output obtained in C).

The key idea is that, although this joint distribution is high-dimensional, the full conditional distributions are simple and available in closed form. In our Bayesian linear regression model with a Normal–Inverse-Gamma prior, the conditional posterior for the regression coefficients(β) is[12]

$$\beta \mid \sigma^2, y, X \sim N\big((M + X^\top X)^{-1} X^\top y,\ \sigma^2 (M + X^\top X)^{-1}\big)\,,$$

However, to implement the Gibbs sampler we also require the full conditional of $\sigma^2 \mid \beta, y, X$.

Starting from Bayes' theorem and treating β as known:

$$p(\sigma^2 \mid \beta, y, X) \propto p(y \mid \beta, \sigma^2) \cdot p(\sigma^2)$$

Substituting

$$\propto (\sigma^2)^{-n/2} \exp\left(-\frac{(y - X\beta)^\top (y - X\beta)}{2\sigma^2}\right) \cdot (\sigma^2)^{-(a+1)} \exp\left(-\frac{b}{\sigma^2}\right)$$

By rearranging the terms we get

$$\propto (\sigma^2)^{-(a+\frac{n}{2}+1)} \exp\left(-\frac{1}{\sigma^2}\left(b + \frac{(y - X\beta)^\top (y - X\beta)}{2}\right)\right)$$

which is the kernel of an Inverse Gamma distribution, giving:

$$\sigma^2 \mid \beta, y, X \sim IG\left(a + \frac{n}{2},\ b + \frac{1}{2}(y - X\beta)^\top (y - X\beta)\right)$$

> Note that unlike the marginal $\sigma^2 \mid y, X$ derived in part B, this full conditional conditions on the current value of β rather than integrating it out, making it considerably simpler to evaluate at each iteration of the sampler.

Thus the conditional posterior for the noise variance is

$$\sigma^2 \mid \beta, y, X \sim \text{IG}\Big(\frac{n}{2} + a,\ b + \tfrac{1}{2}(y - X\beta)^\top (y - X\beta)\Big)\,.$$

The Gibbs sampler alternates between drawing β from $p(\beta \mid \sigma^2, y, X)$ and σ^2 from $p(\sigma^2 \mid \beta, y, X)$ and by repeatedly updating these two blocks in succession, the Markov chain converges to the full posterior distribution, allowing us to approximate $p(\beta, \sigma^2 \mid y, X)$ using the simulated draws.

[12] Here we also use the expression $X^\top X\hat{\beta} = X^\top y$.

```r
gibbs_sampler <- function(y, X, n_iter = 10000,
                          M = diag(ncol(X))/10000,
                          a = 0.0001, b = 0.0001,
                          beta_init = NULL,
                          sigma2_init = 1) {
  n <- length(y)
  p <- ncol(X)

  # Storage
  beta_samples  <- matrix(NA, nrow = n_iter, ncol = p)
  sigma2_samples <- numeric(n_iter)

  # Initial values
  beta_curr <- if (is.null(beta_init)) rep(0, p) else beta_init
  sigma2_curr <- sigma2_init

  # Precompute
  XtX <- t(X) %*% X
  XtY <- t(X) %*% y

  for (t in 1:n_iter) {

    # 1) Sample beta | sigma2, y, X
    # JAGS prior: beta_j ~ N(0, 10000*sigma2)
    # Prior precision = 1/(10000*sigma2)
    M_prior <- diag(p) * (1 / (10000 * sigma2_curr))

    # Posterior precision
    M_post <- XtX + M_prior
    M_post_inv <- solve(M_post)

    beta_mean <- M_post_inv %*% XtY
    beta_cov  <- sigma2_curr * M_post_inv

    beta_curr <- as.vector(MASS::mvrnorm(1,
                     mu = beta_mean, Sigma = beta_cov))

    # 2) Sample sigma2 | beta, y, X
    resid <- y - X %*% beta_curr

    shape_post <- a + n/2
    rate_post  <- b + 0.5 * sum(resid^2)

    sigma2_curr <- 1 / rgamma(1, shape = shape_post, rate = rate_post)

    # Store
    beta_samples[t, ] <- beta_curr
    sigma2_samples[t] <- sigma2_curr
```

```
  }

  list(beta = beta_samples, sigma2 = sigma2_samples)
}
```

The key idea behind the Gibbs sampler is that, although the joint posterior $p(\beta, \sigma^2 \mid y, X)$ is high-dimensional and difficult to sample from directly, the two full conditional distributions are simple and available in closed form. The Gibbs sampler exploits this by alternating between drawing from each full conditional in turn, using the most recently sampled value of the other parameter. Formally, at each iteration t:

$$\beta^{(t)} \sim p(\beta \mid \sigma^{2(t-1)}, y, X)$$

$$\sigma^{2(t)} \sim p(\sigma^2 \mid \beta^{(t)}, y, X)$$

By repeatedly cycling through these two steps, the Markov chain converges to the joint posterior $p(\beta, \sigma^2 \mid y, X)$ as its stationary distribution, and after a sufficient burn-in period the draws can be treated as approximate samples from it.

Step 1: Sample $\beta \mid \sigma^{2(t-1)}, y, X$

From part B, the full conditional for β is:

$$\beta \mid \sigma^2, y, X \sim N_{16}\big((M + X^\top X)^{-1} X^\top y,\ \sigma^2 (M + X^\top X)^{-1}\big)$$

The posterior precision matrix is $M + X^\top X$, where M is the prior precision matrix. Since $M = 10^{-4} \cdot I_{16}$ is very small, the posterior is dominated by the data through $X^\top X$. In the code, the prior precision at iteration t is:

$$M_{\text{prior}} = \frac{1}{10000 \cdot \sigma^{2(t-1)}} I_{16}$$

which matches the JAGS prior $\beta_j \sim N(0, 10000\sigma^2)$. The posterior covariance and mean are then:

$$\Sigma_{\text{post}} = \sigma^{2(t-1)} (M_{\text{prior}} + X^\top X)^{-1}, \qquad \mu_{\text{post}} = (M_{\text{prior}} + X^\top X)^{-1} X^\top y$$

and one draw $\beta^{(t)}$ is taken from this multivariate Normal using `MASS::mvrnorm`.

Step 2: Sample $\sigma^{2(t)} \mid \beta^{(t)}, y, X$

From the derivation above, the full conditional for σ^2 is:

$$\sigma^2 \mid \beta, y, X \sim IG\left(a + \frac{n}{2},\ b + \frac{1}{2}(y - X\beta)^\top (y - X\beta)\right)$$

The updated shape parameter $a + n/2$ adds half the number of observations to the prior shape, while the updated rate parameter adds half the current residual sum of squares $\sum_{i=1}^{n}(y_i - x_i^\top \beta^{(t)})^2$ to the prior rate b. Since R has no built-in inverse gamma sampler, we use the equivalence $\sigma^2 \sim IG(\alpha, \beta) \iff 1/\sigma^2 \sim \text{Gamma}(\alpha, \beta)$ and draw:

$$\tau^{(t)} \sim \text{Gamma}\left(a + \frac{n}{2},\ b + \frac{1}{2}(y - X\beta^{(t)})^\top (y - X\beta^{(t)})\right), \qquad \sigma^{2(t)} = \frac{1}{\tau^{(t)}}$$

At each iteration, $\beta^{(t)}$ is drawn conditioning on the previous $\sigma^{2(t-1)}$, and $\sigma^{2(t)}$ is drawn conditioning on the freshly updated $\beta^{(t)}$. This creates a Markov chain whose transitions leave the joint posterior invariant. Each full conditional draw is exact, so no accept-reject step is needed unlike in Metropolis-Hastings. This is the key practical advantage of Gibbs sampling, whenever the full conditionals are available in closed form and easy to sample from, Gibbs is preferred precisely because it avoids the inefficiency of rejected proposals.

After discarding the first 2000 iterations as burn-in, the remaining draws form a dependent but consistent sample from $p(\beta, \sigma^2 \mid y, X)$.

With the sampler defined, we now apply it to our simulated dataset. We first augment the design matrix with an intercept term and then run the Gibbs algorithm for 10000 iterations, using the same prior hyperparameters as in the JAGS model to ensure comparability.

```
# Add intercept column
X_gibbs <- cbind(1, X_mat)

# Run Gibbs sampler
set.seed(444)

gibbs_out <- gibbs_sampler(
  y = Y,
  X = X_gibbs,
  n_iter = 10000,          # same total iterations
  M = diag(16) / 10000,    # same prior as JAGS
  a = 0.0001,
  b = 0.0001
)
```

We discard the first $2,000$ iterations as burn-in, retaining only the remaining samples for posterior inference.

```r
burn <- 2000

# Burn-in
beta_gibbs  <- gibbs_out$beta[(burn + 1):nrow(gibbs_out$beta), ]
sigma2_gibbs <- gibbs_out$sigma2[(burn + 1):length(gibbs_out$sigma2)]
```

After burn-in, we compute posterior summaries for each parameter. The posterior means serve as Bayesian point estimates for β and σ^2, while the posterior standard deviations quantify their uncertainty, analogous to standard errors in classical regression.

```r
library(knitr)
# TRUE VALUES
beta_true <- c(4.0, 2.0, 0.0, 0.0, 0.0, -1.0, 0.0, 1.5,
               0.0, 0.0, 0.0, 1.0, 0.0, 0.5, 0.0, 0.0)
sigma2_true <- 6.25

# Gibbs summaries
beta_gibbs_mean <- colMeans(beta_gibbs)
beta_gibbs_sd   <- apply(beta_gibbs, 2, sd)
sigma2_gibbs_mean <- mean(sigma2_gibbs)
sigma2_gibbs_sd   <- sd(sigma2_gibbs)

# OLS estimates (same as old table)
ols_fit <- lm(Y ~ X_mat)
beta_ols  <- coef(ols_fit)
ols_sd    <- summary(ols_fit)$coefficients[, "Std. Error"]
sigma2_ols <- summary(ols_fit)$sigma^2

# JAGS posterior means & SDs (same as old table)
beta_jags     <- bayes_summary[, "mean"]
beta_jags_sd  <- bayes_summary[, "sd"]
sigma2_jags    <- fit$BUGSoutput$mean$sigma2
sigma2_jags_sd <- fit$BUGSoutput$sd$sigma2

# Comparison table
comparison_table <- data.frame(
  Parameter     = c(paste0("beta[", 0:15, "]"), "sigma2"),
  True_Value    = c(beta_true, sigma2_true),
  OLS_Mean      = c(beta_ols, sigma2_ols),
  OLS_SD        = c(ols_sd, NA),
  JAGS_Mean     = c(beta_jags, sigma2_jags),
  JAGS_SD       = c(beta_jags_sd, sigma2_jags_sd),
  Gibbs_Mean    = c(beta_gibbs_mean, sigma2_gibbs_mean),
  Gibbs_SD      = c(beta_gibbs_sd, sigma2_gibbs_sd),
  row.names     = NULL
)
kable(
```

```
  comparison_table,
  caption = "Comparison of OLS Estimates, JAGS Posterior Means/SDs,
  and Gibbs Posterior Means/SDs",
  digits = 4,
  align = "lrrrrrrrr"
)
```

Table 7: Comparison of OLS Estimates, JAGS Posterior Means/SDs, and Gibbs Posterior Means/SDs

Parameter	True_Value	OLS_Mean	OLS_SD	JAGS_Mean	JAGS_SD	Gibbs_Mean	Gibbs_SD
beta[0]	4.00	3.5151	0.4249	3.5189	0.3582	3.5155	0.4397
beta[1]	2.00	1.4913	0.4911	1.4929	0.4122	1.4744	0.5007
beta[2]	0.00	-0.0060	0.5959	-0.0058	0.5017	-0.0144	0.6151
beta[3]	0.00	0.1124	0.6993	0.1159	0.5861	0.1038	0.7222
beta[4]	0.00	-0.6202	0.8205	-0.6119	0.6907	-0.6336	0.8490
beta[5]	-1.00	-1.9730	1.3105	-1.9633	1.0995	-1.9914	1.3616
beta[6]	0.00	0.8829	0.4595	0.8764	0.3866	0.8885	0.4770
beta[7]	1.50	1.7844	0.3362	1.7833	0.2837	1.7842	0.3490
beta[8]	0.00	0.7209	0.4018	0.7224	0.3408	0.7210	0.4130
beta[9]	0.00	-0.6623	0.3599	-0.6613	0.3029	-0.6602	0.3682
beta[10]	0.00	0.1342	0.4195	0.1270	0.3538	0.1328	0.4334
beta[11]	1.00	1.4042	0.3607	1.4006	0.3047	1.4134	0.3745
beta[12]	0.00	0.1570	0.4161	0.1510	0.3483	0.1612	0.4330
beta[13]	0.50	-0.1116	0.3309	-0.1100	0.2772	-0.1145	0.3369
beta[14]	0.00	-0.0970	0.3054	-0.0968	0.2578	-0.0925	0.3180
beta[15]	0.00	0.7550	0.3694	0.7555	0.3114	0.7545	0.3826
sigma2	6.25	5.6884	NA	4.0365	0.8516	6.0510	1.5251

It is clear that the means are once again identical across all methods, but the SD's are considerably worse for the custom Gibbs Sampler, before we draw our conclusions, it is meaningful to do some diagnostics:

```
library(coda)
beta1_chain <- gibbs_out$beta[, 1]
mcmc_beta1  <- mcmc(beta1_chain)

par(mfrow = c(2, 1), mar = c(4, 4, 3, 1))
traceplot(mcmc_beta1,
          main = expression(paste("Trace plot – ", beta[1])))
autocorr.plot(mcmc_beta1,
              main = expression(paste("ACF – ", beta[1])),
              auto.layout = FALSE)
```

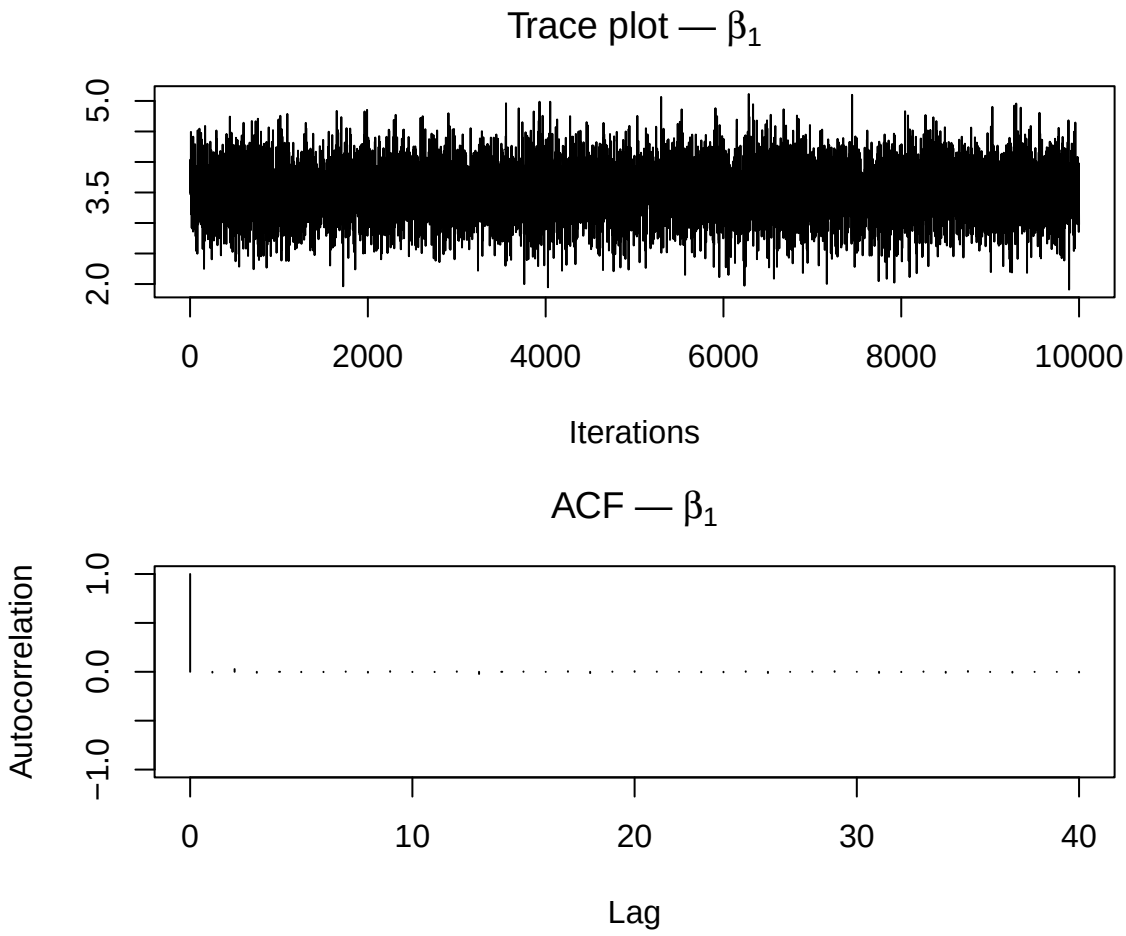

Figure 4: Trace plot (top) and autocorrelation plot (bottom) for β_1 from the Gibbs sampler

```
ess_gibbs <- effectiveSize(mcmc_beta1)
cat(sprintf("Effective Sample Size (ESS): %.0f out of %d nominal draws\n",
            ess_gibbs, nrow(gibbs_out$beta) - 2000))
```

```
Effective Sample Size (ESS): 9614 out of 8000 nominal draws
```

Since the custom Gibbs sampler runs a single chain, the Gelman-Rubin diagnostic $\hat{R}$ is not applicable. We instead assess convergence through the trace plot, autocorrelation plot, and effective sample size for β_1 as a representative parameter.

Conclusion

The trace plot shows a stationary, well-mixed chain from the very first iteration, with no trending or drifting, confirming that the starting values do not affect the posterior draws and no burn-in beyond the discarded 2000 iterations was necessary. The autocorrelation drops to zero immediately after lag 0, indicating that successive samples are essentially independent. The effective sample size of 9614 actually exceeds the nominal 8000 post-burn-in draws, reflecting a slight negative autocorrelation.
Since the custom Gibbs sampler runs a single chain, the Gelman-Rubin diagnostic $\hat{R}$ is not applicable. All diagnostics confirm the sampler is behaving correctly, the slightly larger posterior standard deviations compared to JAGS simply reflect the difference in total sample size: JAGS produced 24000 post-burn-in draws across 3 chains whereas the custom sampler produced 8000 from a single chain, giving three times fewer samples and consequently larger Monte Carlo error in estimating posterior variances.

In short, the sampler is behaving correctly, and the slightly larger posterior standard deviations simply reflect sampling noise rather than a flaw in the algorithm.

Exercise 2

Bayesian Hypothesis Test

Now we are going to perform a Bayesian hypothesis test for a Poisson model. We assume that the daily number of errors on a server follows a Poisson distribution with unknown rate parameter $\lambda > 0$. Over a period of 10 days, we observed a total of 35 errors.
We want to test the following hypotheses:

$$H_0 : \lambda = 2 \quad \text{versus} \quad H_1 : \lambda \neq 2$$

The sufficient statistic for $n = 10$ i.i.d. Poisson observations is $T = \sum_{i=1}^{10} Y_i \sim \text{Poisson}(10\lambda)$, with observed value $T = 35$.

A)

We assume that both hypotheses are equally probable and calculate both marginal likelihoods:

- Marginal likelihood under H_0 ($\lambda = 2$):

$$m_0 = P(T = 35 \mid \lambda = 2) = \frac{e^{-20} \cdot 20^{35}}{35!}$$

- Marginal likelihood under H_1 (Assuming $\lambda \sim \text{Gamma}(2, 1)$), and integrating out λ:

$$m_1 = \int_0^\infty \frac{e^{-10\lambda}(10\lambda)^{35}}{35!} \cdot \lambda e^{-\lambda}\, d\lambda = \frac{10^{35}}{35!} \int_0^\infty \lambda^{36} e^{-11\lambda}\, d\lambda = \frac{10^{35} \cdot \Gamma(37)}{35! \cdot 11^{37}} = \frac{36 \cdot 10^{35}}{11^{37}}$$

since $\int_0^\infty \lambda^{a-1} e^{-b\lambda} = \frac{\Gamma(b)}{b^a}$ from the Gamma Kernel, with $\Gamma(37) = 36!$ and $\Gamma(2) = 1$.

Note that Gamma(2,1)'s mean is $\alpha/\beta = 2$, matching H_0, but its standard deviation is $\sqrt{\alpha/\beta^2} = \sqrt{2} \approx 1.41$, a substantial spread relative to the mean, encoding only mild belief that $\lambda \approx 2$.

To visualize this:

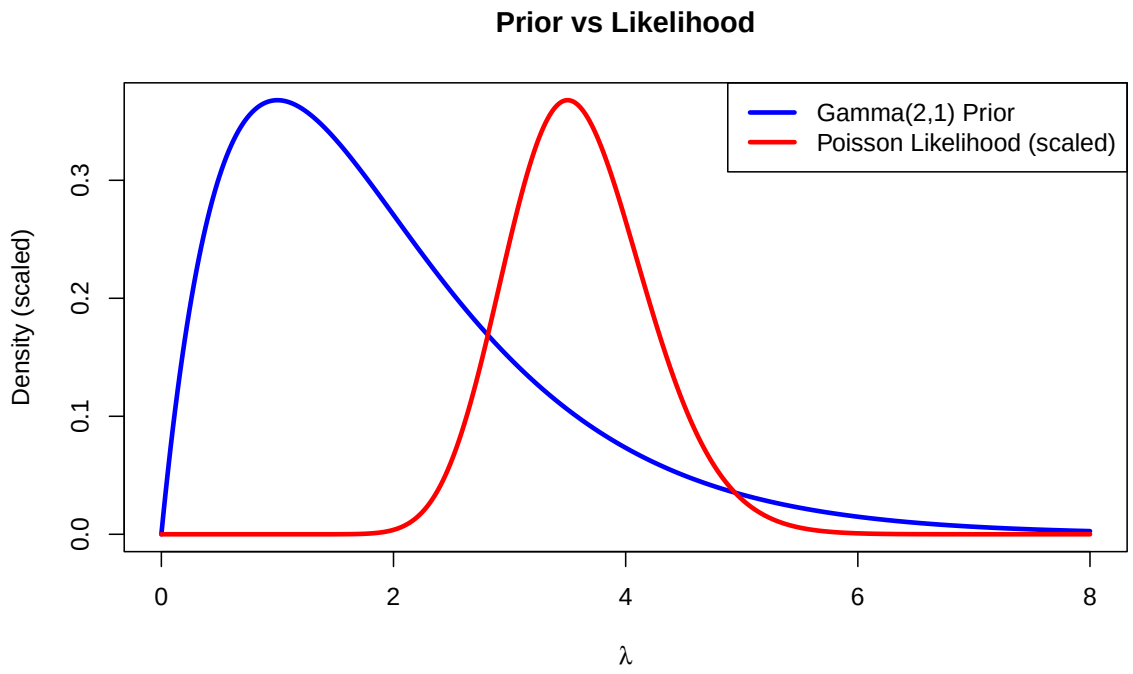

Figure 5: Gamma(2,1) prior (blue) vs Poisson likelihood for T=35 (red). The prior is much flatter, showing it is only weakly informative.

With equal prior probabilities $P(H_0) = P(H_1) = 0.5$[13], the prior odds equal 1, so:

$$BF_{10} = \frac{m_1}{m_0}, \qquad PO_{10} = BF_{10} \times \underbrace{\frac{P(H_1)}{P(H_0)}}_{=1} = BF_{10}$$

This ratio basically indicates how much more likely the observed data are under H_1 than under H_0.

[13] This is a subjective choice reflecting our prior belief about the relative plausibility of the two hypotheses. Assigning $P(H_0) = P(H_1) = 0.5$ is the non-informative option: it expresses a complete absence of prior preference for either hypothesis, letting the data speak through BF_{10} alone.

We compute these numerically below.[14]

```
n       <- 10
T_obs   <- 35
lambda0 <- 2
alpha_prior <- 2; beta_prior <- 1

# Marginal likelihoods (log scale for numerical stability)
log_m0 <- dpois(T_obs, lambda = n * lambda0, log = TRUE)
log_m1 <- log(36) + 35*log(10) - 37*log(11)
#note that normalization cancels out, so these are just the kernels
m0   <- exp(log_m0)
m1   <- exp(log_m1)
BF10 <- m1 / m0
PO10 <- BF10   # equal prior odds => PO = BF
P_H1 <- PO10 / (1 + PO10)
P_H0 <- 1 - P_H1

library(knitr)
kable(data.frame(
  Quantity = c("m0", "m1", "BF10", "PO10",
               "P(H0 | T=35)", "P(H1 | T=35)"),
  Value    = round(c(m0, m1, BF10, PO10, P_H0, P_H1), 4)
), caption = "Bayes Factor and Posterior Odds")
```

Table 8: Bayes Factor and Posterior Odds

Quantity	Value
m0	0.0007
m1	0.0106
BF10	15.4471
PO10	15.4471
P(H0 | T=35)	0.0608
P(H1 | T=35)	0.9392

Conclusion

Since $BF_{10} \approx 15.45 > 10$, this constitutes strong evidence in favour of H_1 on the Jeffreys scale. The posterior probability of H_1 is approximately 0.94, so $\lambda = 2$ is strongly disfavoured given a sample rate of $35/10 = 3.5$.

[14] Note that, the posterior probabilities follow from the odds–probability relation:

$$P(H_1) = \frac{PO_{10}}{1 + PO_{10}}, \quad P(H_0) = 1 - P(H_1).$$

B)

Posterior Predictive Distribution for Y_{11}

We are going to compute the posterior predictive distribution for the 11th observation under the alternative hypothesis. This involves deriving its expected value and variance, comparing these with the classical predictive measures, and finally evaluating the posterior probability that the 11th observation exceeds 5.

Combining the Poisson likelihood with the Gamma$(2, 1)$ prior[15]:

$$\lambda \mid T = 35 \sim \text{Gamma}(\alpha^* = 2 + 35,\ \beta^* = 1 + 10) = \text{Gamma}(37, 11)$$

Marginalising over λ gives us the posterior predictive, the probability of a new observation given the already observed data:

$$P(Y_{11} = k \mid \mathbf{y}) = \int_0^\infty \frac{e^{-\lambda}\lambda^k}{k!} \cdot \frac{11^{37}}{\Gamma(37)} \lambda^{36} e^{-11\lambda}\, d\lambda = \binom{k+36}{k} \left(\frac{11}{12}\right)^{37} \left(\frac{1}{12}\right)^k$$

so $Y_{11} \mid \mathbf{y} \sim \text{NegBin}(r = 37,\ p = 11/12)$.

With the distribution known we can calculate the moments which are given by the Negative Binomial relations:

$$E[Y_{11} \mid \mathbf{y}] = \frac{r(1-p)}{p} = \frac{37 \cdot \frac{1}{12}}{\frac{11}{12}} = \frac{37}{11} \approx 3.364$$

$$\text{Var}[Y_{11} \mid \mathbf{y}] = \frac{r(1-p)}{p^2} = \frac{37 \cdot 12}{121} = \frac{444}{121} \approx 3.669$$

[15] Gamma–Poisson conjugacy: Let $Y_1, \ldots, Y_n \mid \lambda \overset{\text{iid}}{\sim} \text{Poisson}(\lambda)$ and $\lambda \sim \text{Gamma}(\alpha, \beta)$. Then the likelihood is

$$p(Y \mid \lambda) \propto \lambda^{\sum_{i=1}^n Y_i} e^{-n\lambda},$$

and the prior is

$$p(\lambda) \propto \lambda^{\alpha-1} e^{-\beta\lambda}.$$

The posterior is proportional to

$$p(\lambda \mid Y) \propto p(Y \mid \lambda)\, p(\lambda) \propto \lambda^{\alpha-1+\sum Y_i} e^{-(\beta+n)\lambda},$$

which is the kernel of a Gamma$(\alpha + \sum Y_i,\ \beta + n)$ distribution. Thus, the Gamma prior is conjugate to the Poisson likelihood.

Theory:

For the Classical MLE we let $Y_1, \dots, Y_n \overset{\text{iid}}{\sim} \text{Poisson}(\lambda)$. The likelihood then is

$$L(\lambda) = \prod_{i=1}^{n} \frac{e^{-\lambda}\lambda^{Y_i}}{Y_i!},$$

so the log-likelihood is

$$\ell(\lambda) = \sum_{i=1}^{n} (-\lambda + Y_i \log \lambda) + C = -n\lambda + T \log \lambda + C,$$

where $T = \sum_{i=1}^{n} Y_i$. Differentiating and setting to zero,

$$\frac{d\ell}{d\lambda} = -n + \frac{T}{\lambda} = 0 \quad \Rightarrow \quad \hat{\lambda} = \frac{T}{n}.$$

Thus, in classical formulation, the MLE is $\hat{\lambda} = T/n = 3.5$, so the classical plug-in predictive is Poisson(3.5) with mean $=$ variance $= 3.5$. The Bayesian predictive mean (3.364) is slightly lower because the prior pulls the rate toward 2. The Bayesian variance (3.669) exceeds the classical one (3.5) because it propagates uncertainty about λ rather than treating the estimate as exact.

Finally we calculate the posterior probability $P(Y_{11} > 5)$:

$$P(Y_{11} > 5 \mid \mathbf{y}) = 1 - \sum_{k=0}^{5} \binom{k+36}{k} \left(\frac{11}{12}\right)^{37} \left(\frac{1}{12}\right)^{k}$$

```
alpha_post <- alpha_prior + T_obs    # 37
beta_post  <- beta_prior  + n        # 11
r_nb       <- alpha_post
p_nb       <- beta_post / (beta_post + 1)    # 11/12
lambda_mle <- T_obs / n                      # 3.5

pred_mean_bayes <- r_nb * (1 - p_nb) / p_nb
pred_var_bayes  <- r_nb * (1 - p_nb) / p_nb^2
P_gt5_bayes     <- 1 - pnbinom(5, size = r_nb, prob = p_nb)
P_gt5_classical <- 1 - ppois(5, lambda = lambda_mle)

kable(data.frame(
  Quantity = c("Predictive mean", "Predictive variance",
               "P(Y11 > 5)"),
  Bayesian = round(c(pred_mean_bayes, pred_var_bayes, P_gt5_bayes), 4),
  Classical = round(c(lambda_mle, lambda_mle, P_gt5_classical), 4)
), caption = "Bayesian vs classical predictive summaries")
```

Table 9: Bayesian vs classical predictive summaries

Quantity	Bayesian	Classical
Predictive mean	3.3636	3.5000
Predictive variance	3.6694	3.5000
P(Y11 > 5)	0.1336	0.1424

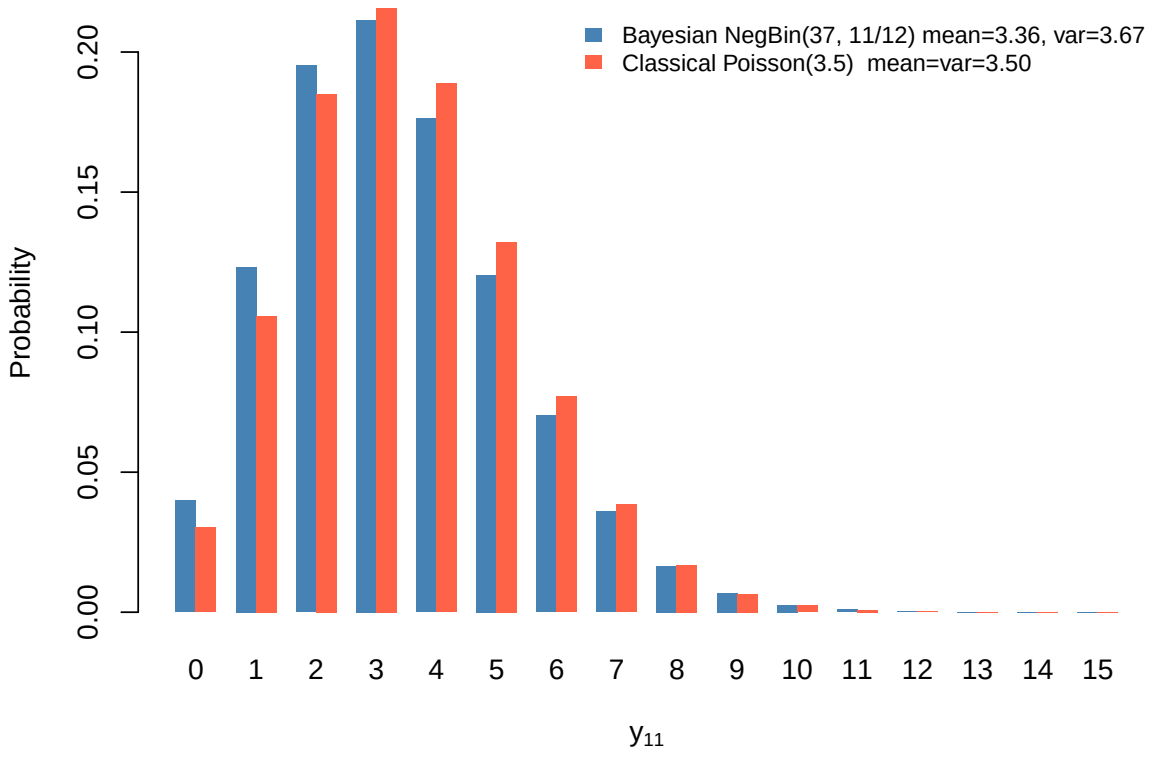

Figure 6: Predictive distribution for Y_{11}

Conclusion

The Bayesian $P(Y_{11} > 5 \mid \mathbf{y}) \approx 0.1336$ is slightly lower than the classical 0.1424, consistent with the prior pulling the mean below the MLE of 3.5. Thus it is less probable to have higher values overall.

Exercise 3

Custom Exotic Star Dataset Analysis

We will generate a dataset of 163 stars drawn from a spectroscopic survey, classified into five spectral categories: O, B, A, F, and G, ordered from hottest to coolest surface temperature. For each star, the binary response variable indicates whether the star exhibits exotic behavior (e.g. anomalous emission, pulsar candidacy, or unusual spectral activity).

The dataset contains 20 type-O stars, 35 type-B, 40 type-A, 38 type-F, and 30 type-G stars, with observed exotic rates decreasing from class O down to class G. That clear decreasing trend in the probability of exotic behavior is observed as surface temperature decreases, which is physically motivated by the fact that hotter, more massive stars undergo more extreme internal processes. The explanatory variable is therefore the spectral class, a categorical variable with 5 categories, and the response variable follows a Binomial distribution.

For the type-O class specifically, we assume that a previous independent survey observed 6 exotic stars out of 15, and we incorporate this historical information into an informative Beta$(7, 10)$ prior for the corresponding success probability θ_O, while all remaining classes receive a non-informative Beta$(1, 1)$ prior.

For each spectral class $k \in \{O, B, A, F, G\}$, let y_k denote the number of exotic stars observed out of n_k total stars. We model[16]:

$$y_k \mid \theta_k \sim \text{Binomial}(n_k,\, \theta_k), \qquad k = 1, \ldots, 5$$

where $\theta_k \in (0, 1)$ is the unknown probability of a star in class k being exotic.

Theory: Prior Distributions

The prior distributions are chosen as follows. For class O, we have historical data from a past experiment (6 exotic out of 15), which we encode as an informative Beta prior. For all other classes, we use a flat non-informative prior:

$$\theta_O \sim \text{Beta}(7,\, 10), \qquad \theta_k \sim \text{Beta}(1,\, 1) \text{ for } k \in \{B, A, F, G\}.$$

The Beta$(7, 10)$ prior has mean $7/17 = 0.412$ and encodes the belief from the past experiment that roughly 40% of type-O stars are exotic. The Beta$(1, 1)$ is the uniform distribution on $(0, 1)$, conveying no prior preference for any value of θ_k.

[16] Of course, here Binomial is selected due to the data's target variable being binary(0,1; success or failure)

Since the Beta distribution is conjugate to the Binomial likelihood(that is also why we chose Betas as priors), the posterior distributions are available in closed form:

$$\theta_k \mid y_k \sim \text{Beta}(\alpha_k + y_k,\ \beta_k + n_k - y_k)$$

where (α_k, β_k) are the prior hyperparameters. Explicitly:

$$\theta_O \mid y_O \sim \text{Beta}(7 + y_O,\ 10 + n_O - y_O), \qquad \theta_k \mid y_k \sim \text{Beta}(1 + y_k,\ 1 + n_k - y_k) \text{ for } k \neq O.$$

We first load the dataset and compute group-level summaries.

```
# We actually generate the dataset with the same seed every time
# Since there is no astrophysical dataset this short.
set.seed(42)
is_exotic <- c(
  rbinom(20, 1, 0.45),    # Class O
  rbinom(35, 1, 0.25),    # Class B
  rbinom(40, 1, 0.15),    # Class A
  rbinom(38, 1, 0.08),    # Class F
  rbinom(30, 1, 0.05)     # Class G
)
spectral_class <- factor(
  rep(c("O","B","A","F","G"), c(20,35,40,38,30)),
  levels = c("O","B","A","F","G")
)
dat <- data.frame(spectral_class = spectral_class,
                  is_exotic = is_exotic)
# Group-level aggregates used throughout
groups <- c("O","B","A","F","G")
y_vec  <- as.integer(tapply(dat$is_exotic, dat$spectral_class,
                            sum)[groups])
n_vec  <- as.integer(tapply(dat$is_exotic, dat$spectral_class,
                            length)[groups])
library(knitr)
kable(
  data.frame(
    `Spectral Class` = groups,
    `n (total)`      = n_vec,
    `y (exotic)`     = y_vec,
    `p-hat`          = round(y_vec / n_vec, 3),
    check.names      = FALSE
  ),
  caption = "Observed data by spectral class"
)
```

Table 10: Observed data by spectral class

Spectral Class	n (total)	y (exotic)	p-hat
O	20	12	0.600
B	35	13	0.371
A	40	3	0.075
F	38	3	0.079
G	30	0	0.000

The observed exotic rates show a clear decreasing trend with spectral class, from $\hat{p}_O = 0.600$ for the hottest type-O stars down to $\hat{p}_G = 0.000$ for the coolest type-G stars. Type-O stars are by far the most likely to exhibit exotic behavior, while classes F and G show very low or zero exotic activity. The total sample size is $N = 163$ stars across the five groups.

We use JAGS to sample from the posterior distributions of all five θ_k parameters and we are going to apply a 2000 sample burn-in followed by 10000 iterations, in 3 independent chains, giving $3 \times 10000 = 30000$ posterior samples per θ_k.

> Since only θ_O carries an informative prior with mean $7/17 = 0.412$, which is below the observed $\hat{p}_O = 0.600$, we expect its posterior mean to be pulled downward away from the raw estimate. All other classes have flat Beta$(1, 1)$ priors and are therefore expected to have posterior means very close to their observed $\hat{p}_k$.

```
suppressPackageStartupMessages(library(R2jags))


jags_data <- list(y = y_vec, n = n_vec, K = 5)


jags_inits  <- function() list(theta = rep(0.3, 5))

jags_params <- c("theta")


# Model: Beta-Binomial with mixed priors
# theta[1] = class O  -> informative Beta(7, 10)
# theta[2:5]          -> non-informative Beta(1, 1)
jags_model <- "
model {
  for (k in 1:K) {
    y[k] ~ dbin(theta[k], n[k])
  }
  theta[1] ~ dbeta(7, 10)
  for (k in 2:K) {
    theta[k] ~ dbeta(1, 1)
```

```
  }
}
"

model_file <- file.path(tempdir(), "model_ex3.txt")

writeLines(jags_model, model_file)

fit3 <- jags(
  data               = jags_data,
  inits              = jags_inits,
  parameters.to.save = jags_params,
  model.file         = model_file,
  n.chains           = 3,
  n.iter             = 12000,
  n.burnin           = 2000,
  n.thin             = 1
)
```

Before interpreting the results we assess convergence to ensure the posterior samples are reliable. We use three complementary diagnostics.[17]

First, trace plots to show the sampled values of each θ_k across iterations for all 3 chains. Well-mixed chains should overlap and show no trending or drifting, producing the characteristic "hairy caterpillar'' pattern.

```
library(coda)
mcmc3 <- as.mcmc(fit3)

# Name by matching JAGS names
class_labels <- c("theta[O]","theta[B]","theta[A]","theta[F]",
                  "theta[G]")

for (i in seq_along(mcmc3)) {
  old_names <- colnames(mcmc3[[i]])
  for (j in 1:5) {
    old_names[old_names == paste0("theta[", j, "]")] <- class_labels[j]
  }
  colnames(mcmc3[[i]]) <- old_names
}

par(mfrow = c(5, 2), mar = c(3, 3, 2, 1))
for (k in class_labels) {
  plot(mcmc3[, k], main = k, auto.layout = FALSE)
}
```

[17] Note that, the diagnostics are identical to the ones we used earlier.

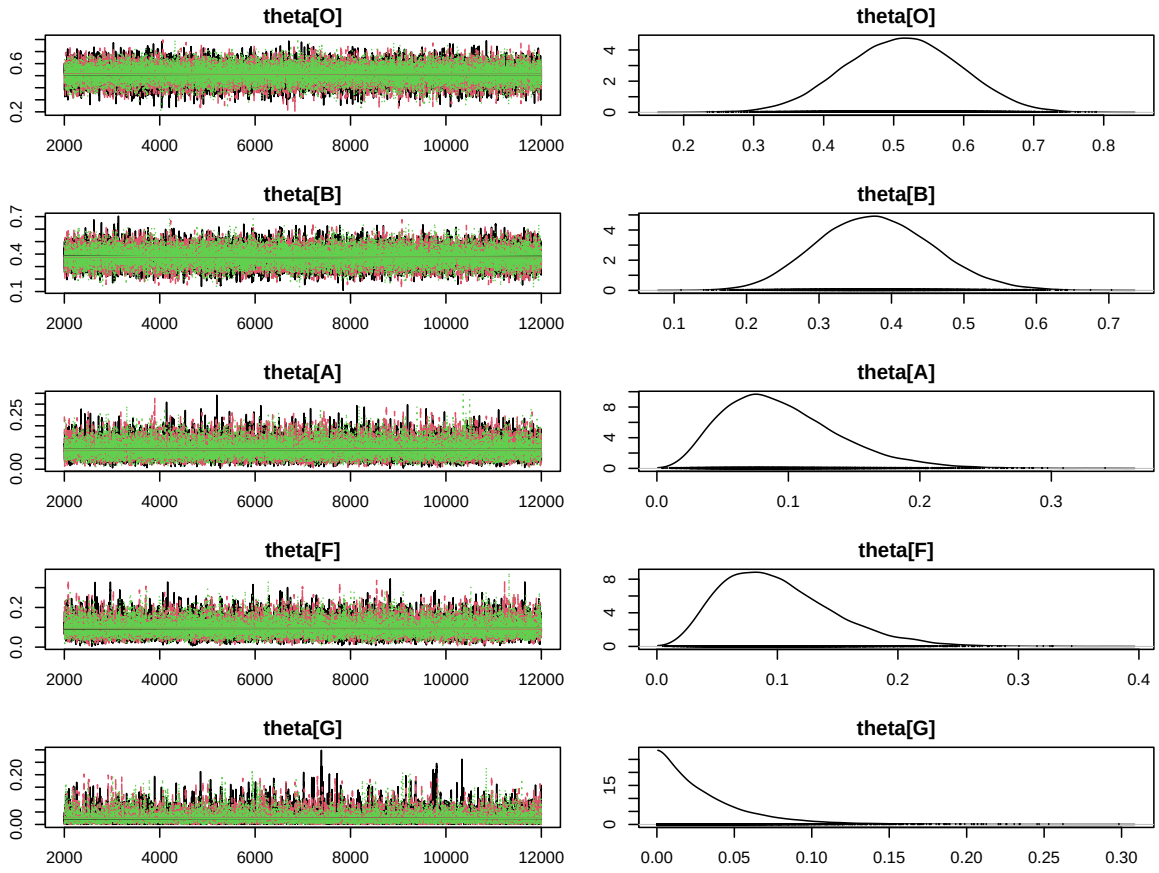

Figure 7: Trace plots (left) and posterior density estimates (right) for all five θ_k parameters.

All five trace plots display the characteristic stationary, well-mixed "hairy caterpillar'' pattern across all 3 chains (black, red, green), with no trending, drifting, or getting stuck at any point. This confirms that the burn-in of 2000 iterations was sufficient and all chains converged to the same stationary posterior distribution.

Notably, θ_O is centered around ≈ 0.51, visibly pulled below the observed $\hat{p}_O = 0.600$ toward the informative prior mean of 0.4, confirming the prior is doing its job. The remaining parameters are centered close to their observed rates: $\theta_B \approx 0.38$, $\theta_A \approx 0.09$, $\theta_F \approx 0.10$, and $\theta_G \approx 0.03$, all consistent with their flat Beta$(1, 1)$ priors exerting negligible influence on the posterior.

We move on to the Gelman–Rubin statistic $\hat{R}$ compares the variance within chains to the variance between chains; values close to 1 indicate that all chains have converged to the same distribution, with $\hat{R} < 1.1$ being the standard threshold.

```
gelman3    <- gelman.diag(mcmc3[, class_labels])


gelman_df3 <- data.frame(
  Parameter = class_labels,
  Point_Est = round(gelman3$psrf[, 1], 4),
  Upper_CI  = round(gelman3$psrf[, 2], 4),
  row.names = NULL
)


kable(gelman_df3, caption = "Gelman-Rubin convergence
      diagnostic ($\\hat{R}$)")
```

Table 11: Gelman–Rubin convergence diagnostic ($\hat{R}$)

Parameter	Point_Est	Upper_CI
theta[O]	1.0004	1.0014
theta[B]	1.0000	1.0001
theta[A]	1.0007	1.0016
theta[F]	1.0005	1.0011
theta[G]	1.0042	1.0069

All $\hat{R}$ values are extremely close to 1, well below the standard threshold of 1.1, confirming that all 3 chains have converged to the same stationary distribution.

Lastly, the autocorrelation plots.

```
par(mfrow = c(5, 1), mar = c(3, 4, 2, 1))

for (k in class_labels) {
  autocorr.plot(mcmc3[[1]][, k], main = k, auto.layout = FALSE)

}
```

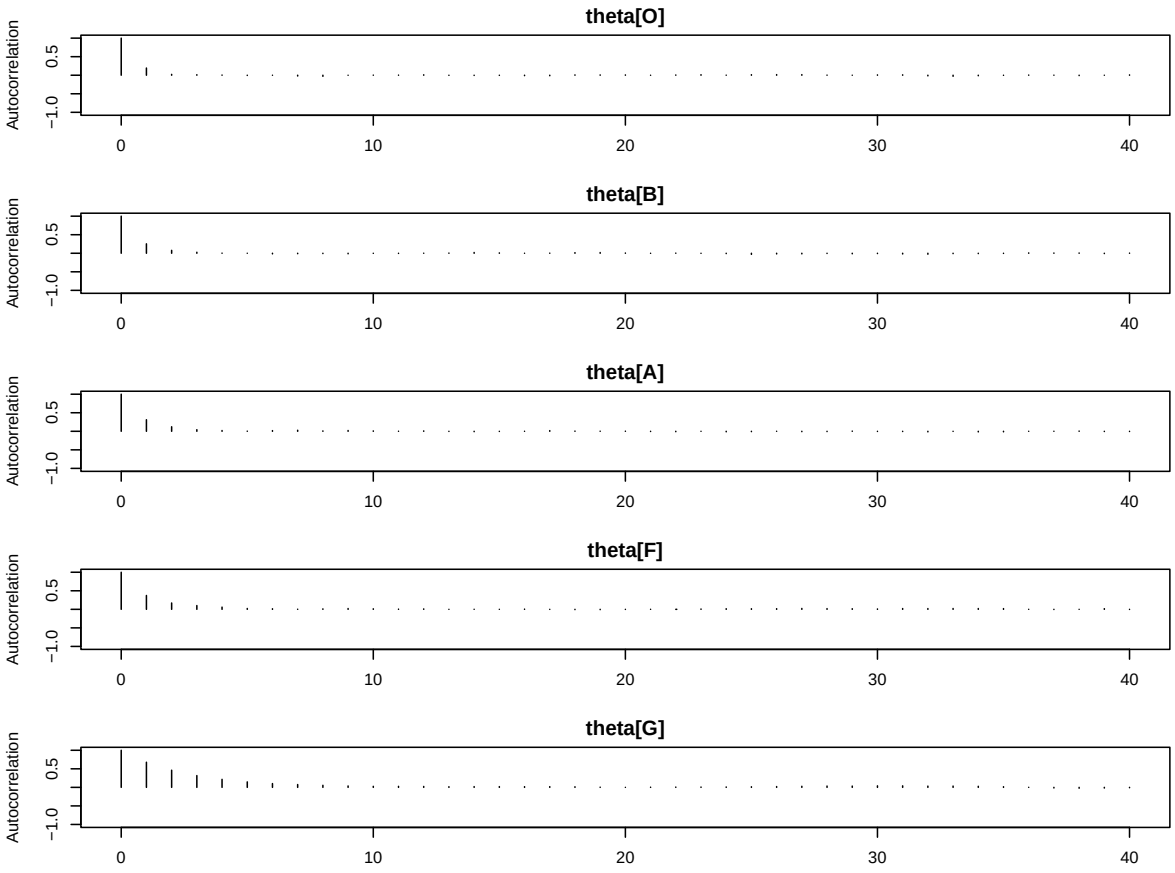

Figure 8: Autocorrelation plots for all five parameters from chain 1.

All five parameters show a mild autocorrelation at lag 1, ranging from approximately 0.2–0.3 for θ_O, θ_B, θ_A, and θ_F, before dropping to zero by lag 2. θ_G shows the most persistent autocorrelation, with a lag-1 value of approximately 0.6 that gradually decays to zero by around lag 6–7. This is expected given that its posterior is concentrated very close to zero, as all 30 type-G stars were non-exotic, leaving the chain less room to explore. Overall the autocorrelation structure is mild and short-lived for all parameters, indicating that the 30000 posterior samples are close to independent and no convergence issues are present.

Given the short-lived autocorrelation structure across all parameters, no thinning is required, discarding samples would reduce the effective sample size without any meaningful reduction in autocorrelation.

We can now confidently see the full results from the posteriors.

```
bayes3 <- fit3$BUGSoutput$summary[paste0("theta[", 1:5, "]"), ]
kable(
  data.frame(
    Parameter  = class_labels,
```

```
    Mean       = round(bayes3[, "mean"],  4),
    SD         = round(bayes3[, "sd"],    4),
    `2.5%`     = round(bayes3[, "2.5%"], 4),
    `97.5%`    = round(bayes3[, "97.5%"],4),
    check.names = FALSE,
    row.names     = NULL
  ),

  caption = "Posterior summary for $\\theta_k$ — all
  five spectral classes")
```

Table 12: Posterior summary for θ_k — all five spectral classes

Parameter	Mean	SD	2.5%	97.5%
theta[O]	0.5124	0.0814	0.3535	0.6688
theta[B]	0.3783	0.0792	0.2317	0.5404
theta[A]	0.0950	0.0441	0.0277	0.1976
theta[F]	0.1002	0.0469	0.0290	0.2100
theta[G]	0.0312	0.0307	0.0008	0.1144

We compare the Bayesian posterior means with the observed rates, which in this saturated binomial model are equivalent to the classical estimates[18].

```
kable(
  data.frame(
    Class            = groups,

    `Observed p-hat` = round(y_vec / n_vec, 4),

    `Classical SD`   = round(sqrt((y_vec/n_vec) * (1
                        - y_vec/n_vec) / n_vec), 4),

    `Bayesian Mean`  = round(bayes3[, "mean"], 4),

    `Bayesian SD`    = round(bayes3[, "sd"],   4),

    check.names      = FALSE,

    row.names     = NULL
  ),
  caption = "Observed rates vs Bayesian posterior means"
)
```

[18] That's literally all classical statistics does for this problem count successes and divide by total, which we already did when generating the data.

Table 13: Observed rates vs Bayesian posterior means

Class	Observed p-hat	Classical SD	Bayesian Mean	Bayesian SD
O	0.6000	0.1095	0.5124	0.0814
B	0.3714	0.0817	0.3783	0.0792
A	0.0750	0.0416	0.0950	0.0441
F	0.0789	0.0437	0.1002	0.0469
G	0.0000	0.0000	0.0312	0.0307

Since the Beta–Binomial model is conjugate, the posterior distributions are available in closed form without any MCMC sampling. Specifically, for each spectral class k:

$$\theta_k \mid y_k \;\sim\; \text{Beta}(\alpha_k + y_k,\; \beta_k + n_k - y_k),$$

where (α_k, β_k) are the prior hyperparameters. We can therefore plot the exact posterior densities by evaluating the Beta density on a fine grid, using the updated parameters $(\alpha_k + y_k,\;\; \beta_k + n_k - y_k)$ directly.

We plot these five posterior densities together and then make a forest plot for comparisons, placing the Bayesian posterior mean with its 95% credible interval alongside the classical MLE with its 95% confidence interval for every class.

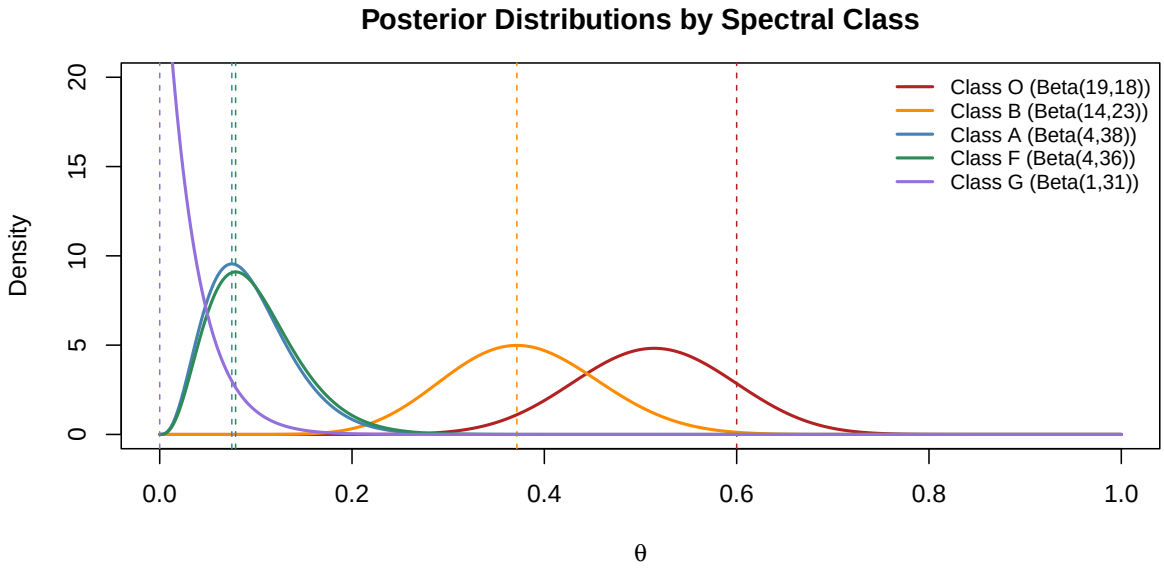

Figure 9: Posterior Beta distributions for θ_k across all spectral classes. The dashed vertical lines mark the observed $\hat{p}_k$.

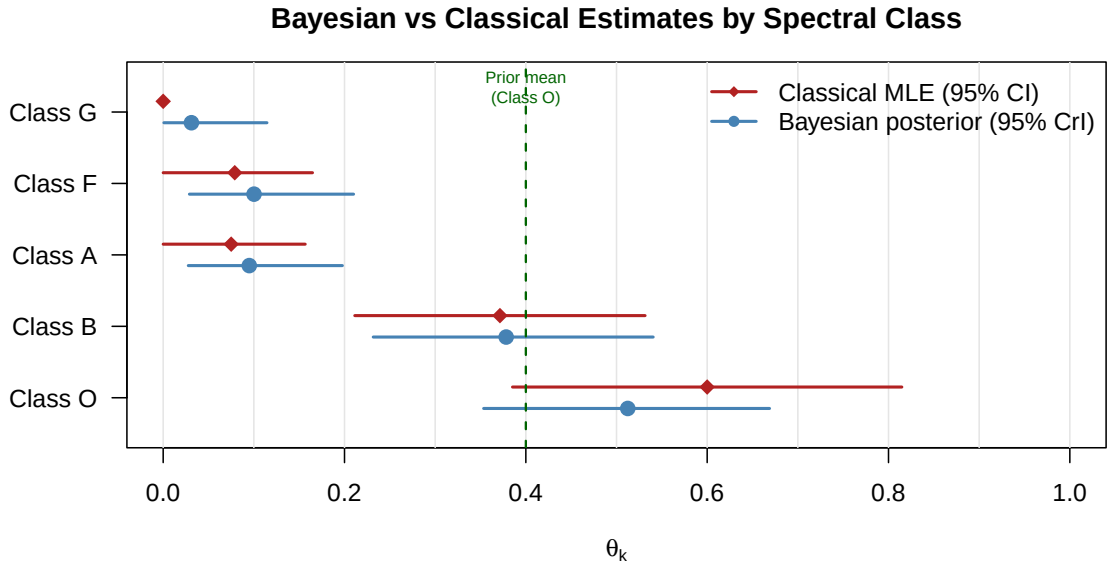

Figure 10: Forest plot comparing Bayesian posterior means (95% credible intervals, blue) against classical MLE estimates (95% confidence intervals, red) for each spectral class.

Conclusion

Class O is the most instructive case. The informative Beta$(7, 10)$ prior, encoding the historical experiment of 6 exotic stars out of 15, pulls the posterior mean down to 0.5124 from the observed MLE of 0.600. This shrinkage is clearly visible in both Figure 9 and in Figure 10, where the blue Bayesian point lies noticeably to the left of the red classical point. At the same time, the posterior credible interval $(0.3535, 0.6688)$ is narrower than the classical confidence interval, reflecting the additional information contributed by the prior.

Classes B, A, and F carry flat Beta$(1, 1)$ priors, so the posterior is driven entirely by the data. The Bayesian and classical estimates are nearly identical, with credible intervals and confidence intervals of comparable width, as expected when the prior is flat and the sample sizes are moderate.

Class G is a special limiting case. All 30 type-G stars were non-exotic, so the MLE is exactly $\hat{p}_G = 0$ and the Wald formula collapses to a degenerate point interval of zero width, visible in Figure 10 as the red point with no error bar. The Bayesian approach avoids this entirely: the Beta$(1, 1)$ prior prevents the posterior from collapsing to a point mass at zero, yielding a proper posterior Beta$(1, 31)$ with mean 0.0316 and a finite credible interval $(0.0008,\ 0.1144)$. This is one of the clearest practical advantages of Bayesian inference over classical methods in small or extreme samples.

Overall, the Bayesian model correctly captures the physically motivated decreasing trend in exotic activity from class O down to class G, incorporates historical information where available, and produces well-behaved uncertainty estimates even in the degenerate case where classical inference breaks down.

Exercise 4

Metropolis–Hastings for the Jeffreys Prior

We return to the Poisson server-error problem of Exercise 2. Under the alternative hypothesis $H_1 : \lambda \neq 2$, we are going to derive the Jeffreys prior for λ analytically, then implement a Metropolis–Hastings (MH) algorithm in R to simulate 20 000 draws from the resulting posterior. We will use a Normal proposal distribution centered at the current chain value with a variance tuned to achieve approximately 25% acceptance.

Theory: Jeffreys Prior

The Jeffreys prior is the unique (up to normalization) prior that is invariant under re-parametrisation of the model. It is defined as

$$p_J(\lambda) \propto \sqrt{I(\lambda)},$$

where $I(\lambda)$ is the Fisher information:

$$I(\lambda) = -\mathbb{E}\left[\frac{\partial^2 \log p(Y \mid \lambda)}{\partial \lambda^2}\right]$$

The likelihood for a single Poisson observation Y is:[19]

$$p(Y \mid \lambda) = \frac{e^{-\lambda}\lambda^Y}{Y!}$$

Thus, the log-likelihood is

$$\log p(Y \mid \lambda) = -\lambda + Y \log \lambda - \log(Y!)$$

Differentiating twice with respect to λ:

$$\frac{\partial \log p}{\partial \lambda} = -1 + \frac{Y}{\lambda}, \qquad \frac{\partial^2 \log p}{\partial \lambda^2} = -\frac{Y}{\lambda^2}.$$

Taking the expectation (recalling $\mathbb{E}[Y] = \lambda$ for Poisson, and λ is just a constant):

$$I(\lambda) = -\mathbb{E}\left[-\frac{Y}{\lambda^2}\right] = \frac{\mathbb{E}[Y]}{\lambda^2} = \frac{\lambda}{\lambda^2} = \frac{1}{\lambda}.$$

[19] In this case a single Poisson observation refers to the amount of server lags in a single day, we could generalize this by assuming n days worth of observations but since that would just follow a Poisson$(n\lambda)$, n would just become part of the normalization constant and not really affect the posterior's kernel.

The Jeffreys prior for the Poisson rate parameter is therefore

$$\boxed{p_J(\lambda) \propto \sqrt{\frac{1}{\lambda}} = \lambda^{-1/2}, \qquad \lambda > 0.}$$

This is an improper prior (it does not integrate to a finite constant over $(0, \infty)$), but it yields a proper posterior as shown below, so it remains valid for inference.

The sufficient statistic is again $T = \sum_{i=1}^{10} Y_i = 35$, with $T \mid \lambda \sim \text{Poisson}(10\lambda)$. Combining the likelihood with the Jeffreys prior to get the posterior:

$$p(\lambda \mid T) \;\propto\; \underbrace{e^{-10\lambda}(10\lambda)^{35}}_{\text{likelihood}} \times \underbrace{\lambda^{-1/2}}_{\text{Jeffreys prior}} \;=\; e^{-10\lambda}\,\lambda^{35.5-1}.$$

This is the kernel of a $\text{Gamma}(35.5,\ 10)$ distribution, confirming propriety:

$$\lambda \mid T = 35 \;\sim\; \text{Gamma}(35.5,\ 10).$$

The posterior mean is $35.5/10 = 3.55$ and the posterior standard deviation is $\sqrt{35.5}/10 \approx 0.596$.

The Jeffreys posterior mean (3.55) is very close to the MLE $\hat{\lambda} = 3.5$, as expected for a non-informative prior with $n = 10$ observations. It is also slightly higher than the $\text{Gamma}(2, 1)$ posterior mean because the $\text{Gamma}(2, 1)$ prior was centered at 2, actively pulling the posterior toward the null value, whereas the Jeffreys prior has no such concentration and therefore lets the data dominate more freely.

We implement the Metropolis–Hastings algorithm using a symmetric Normal proposal $q(\lambda^* \mid \lambda^{(t)}) = N(\lambda^{(t)},\ \sigma^2_{\text{prop}})$.

At each iteration t:

1. Propose $\lambda^* \sim N(\lambda^{(t)},\ \sigma^2_{\text{prop}})$.
2. If $\lambda^* \leq 0$, reject immediately (the target has support on $(0, \infty)$).
3. Otherwise compute the log acceptance ratio

$$\log r = \log p(\lambda^* \mid T) - \log p(\lambda^{(t)} \mid T) = \left(T - \tfrac{1}{2}\right) \log \frac{\lambda^*}{\lambda^{(t)}} - n\left(\lambda^* - \lambda^{(t)}\right).$$

4. Set $\lambda^{(t+1)} = \lambda^*$ with probability $\min(1,\ e^{\log r})$, otherwise $\lambda^{(t+1)} = \lambda^{(t)}$.

Because the Normal proposal is symmetric ($q(\lambda^* \mid \lambda) = q(\lambda \mid \lambda^*)$), the proposal ratio cancels in the acceptance probability, leaving only the target ratio. This is the original Metropolis algorithm, a special case of MH.

We need to tune the variance of the proposed normal distribution in such a way that only 25% of the proposed λ are accepted. Here $\sigma_{\text{post}} \approx 0.596$, so we start with that and adjust until we succeed.

```r
set.seed(444)

n_iter   <- 20000
T_obs    <- 35
n_days   <- 10
prop_sd  <- 2.8

log_post_jeffreys <- function(lam) {
  if (lam <= 0) return(-Inf)
  (T_obs - 0.5) * log(lam) - n_days * lam
}

chain         <- numeric(n_iter)
chain[1]      <- T_obs / n_days
accept_count  <- 0

for (i in 2:n_iter) {
  lam_curr <- chain[i - 1]
  lam_prop <- rnorm(1, mean = lam_curr, sd = prop_sd)
  log_r <- log_post_jeffreys(lam_prop) - log_post_jeffreys(lam_curr)
  if (log(runif(1)) < log_r) {
    chain[i]     <- lam_prop
    accept_count <- accept_count + 1
  } else {
    chain[i] <- lam_curr
  }
}
accept_rate <- accept_count / (n_iter - 1)
cat(sprintf("Acceptance rate: %.3f  (%.1f%%)\n",
            accept_rate, 100 * accept_rate))
```

```
Acceptance rate: 0.254  (25.4%)
```

We ultimately increased σ_{prop} to 2.8 to achieve the target 25% acceptance rate. A larger proposal SD means candidates are drawn from a wider neighborhood around the current value, so they more frequently land in low-probability regions of the posterior and get rejected. This is intentional , as it prevents the chain from accepting nearly every proposal and taking steps so small that it barely explores the parameter space.

Next, we apply the burn-in:

```
post_mean_exact <- 35.5 / 10
burn_in    <- 2000
chain_post <- chain[(burn_in + 1):n_iter]
n_post     <- length(chain_post)
```

We discard the first 2,000 iterations as burn-in. The trace ergodic mean plots below show that the chain reaches stationarity well within the first $\sim$ 1500 iterations given initialization at the MLE, so 2,000 is conservative and more than sufficient.

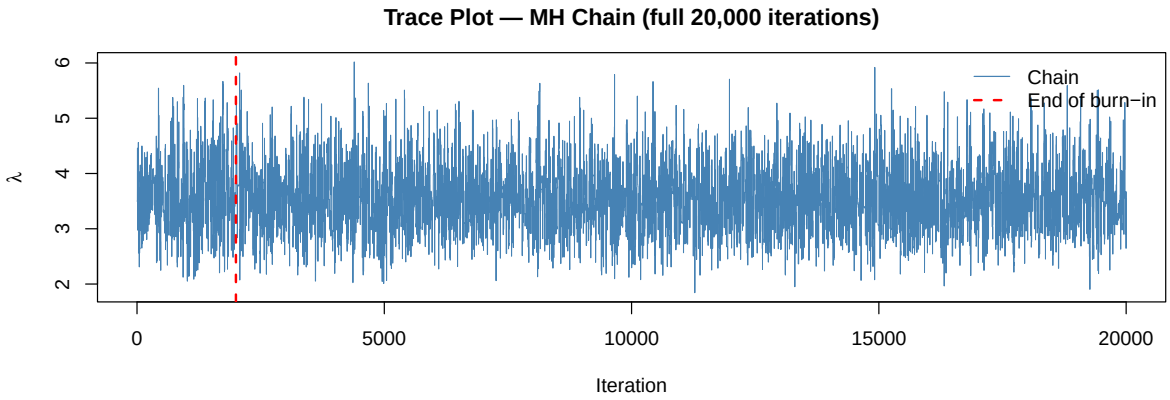

Figure 11: Trace plot for the full 20,000-iteration MH chain. The red dashed line marks the end of burn-in (iteration 2,000).

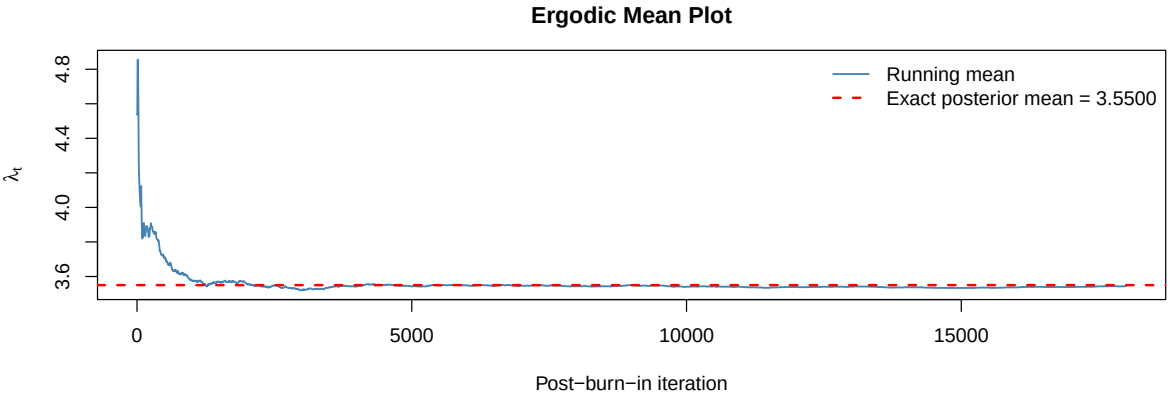

Figure 12: Ergodic mean plot. The running mean converges rapidly to the exact posterior mean of 3.55 (red dashed line).

We also check if the amount of samples(20.000) is sufficient, so once again we do that by doing autocorrelation plots and calculating the Effective Sample Size(ESS).

```
library(coda)
mcmc_chain <- mcmc(chain_post)
par(mar = c(4, 4, 3, 1))
autocorr.plot(mcmc_chain,
              main = expression("ACF — post-burn-in " * lambda * " chain"),
              auto.layout = FALSE)
```

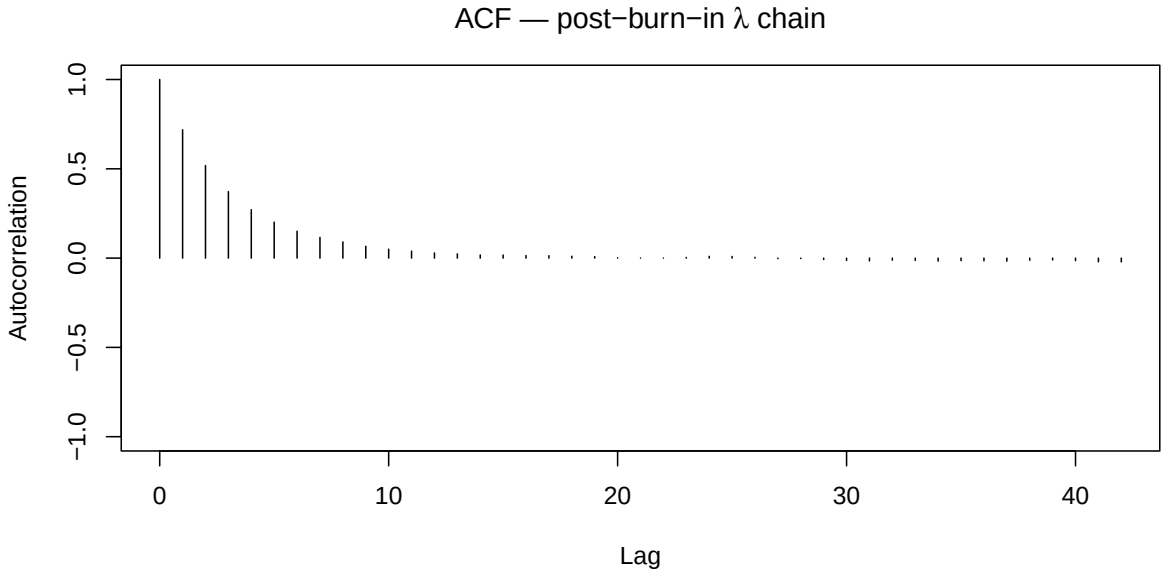

Figure 13: Autocorrelation function of the post-burn-in MH chain up to lag 40.

```
eff_n <- effectiveSize(mcmc_chain)
cat(sprintf("Effective Sample Size (ESS): %.0f out of %d nominal draws\n",
            eff_n, n_post))
```

```
Effective Sample Size (ESS): 2947 out of 18000 nominal draws
```

```
cat(sprintf("ESS ratio: %.3f\n", eff_n / n_post))
```

```
ESS ratio: 0.164
```

With a Normal random-walk proposal and $\sim 25\%$ acceptance, some positive autocorrelation at low lags is expected. The ACF decays to zero within ~10-15 lags and the ESS is well above 1 000, so no thinning is required.

With the diagnostics completed, we determine that the model works effectively, it doesn't get stuck or repeats itself, the burn-in is sufficient, the sample size is big enough and the entire space is mapped. Thus we present the descriptive summary table with confidence:

```
library(knitr)

post_mean_exact <- 35.5 / 10
post_sd_exact   <- sqrt(35.5) / 10
post_med_exact  <- qgamma(0.5,   shape = 35.5, rate = 10)
post_q025_exact <- qgamma(0.025, shape = 35.5, rate = 10)
post_q975_exact <- qgamma(0.975, shape = 35.5, rate = 10)

sim_mean   <- mean(chain_post)
sim_sd     <- sd(chain_post)
sim_median <- median(chain_post)
sim_q025   <- quantile(chain_post, 0.025)
sim_q975   <- quantile(chain_post, 0.975)

kable(data.frame(
  Quantity     = c("Mean", "SD", "Median",
                   "2.5th percentile", "97.5th percentile"),
  Exact_Gamma  = round(c(post_mean_exact, post_sd_exact, post_med_exact,
                         post_q025_exact, post_q975_exact), 4),
  MH_Simulated = round(c(sim_mean, sim_sd, sim_median,
                         sim_q025, sim_q975), 4)
), caption = "Descriptive summary: exact Gamma(35.5, 10) vs MH simulated values")
```

Table 14: Descriptive summary: exact Gamma(35.5, 10) vs MH simulated values

Quantity	Exact_Gamma	MH_Simulated
Mean	3.5500	3.5437
SD	0.5958	0.5903
Median	3.5167	3.5278
2.5th percentile	2.4796	2.4914
97.5th percentile	4.8094	4.7974

For some visualization of how the MCMC algorithm performed, we compare the histogram of the generated values to the actual posterior we calculated analytically.

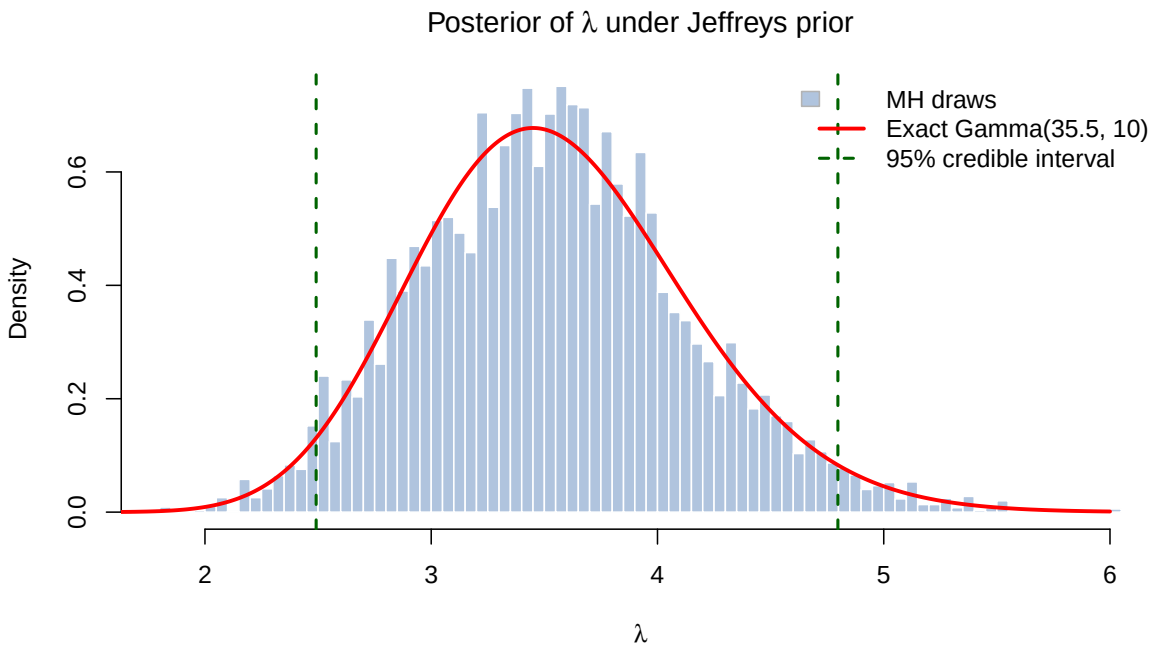

Figure 14: Histogram of the 18,000 post-burn-in MH draws overlaid with the exact Gamma(35.5, 10) density (red). Green dashed lines mark the 95% credible interval.

Finally using our generated values we can calculate the 95% credible intervals(CIs) and compare them to the fully analytical ones.

```
ci_sim   <- quantile(chain_post, c(0.025, 0.975))

ci_exact <- qgamma(c(0.025, 0.975), shape = 35.5, rate = 10)

kable(data.frame(
  Method  = c("MH simulation", "Exact Gamma(35.5, 10)"),
  Lower   = round(c(ci_sim[1],   ci_exact[1]), 4),
  Upper   = round(c(ci_sim[2],   ci_exact[2]), 4),
  Width   = round(c(diff(ci_sim), diff(ci_exact)), 4)
),
caption = "Symmetric 95\\% credible interval for $\\lambda$")
```

Table 15: Symmetric 95% credible interval for λ

	Method	Lower	Upper	Width
2.5%	MH simulation	2.4914	4.7974	2.3060
	Exact Gamma(35.5, 10)	2.4796	4.8094	2.3298

Conclusion

Credible interval. The symmetric 95% credible interval for λ under the Jeffreys prior is approximately $\lambda \in (2.48,\ 4.70)$ (exact analytical values; the MH interval agrees to within Monte Carlo error).

Interpretation. Given the observed data ($T = 35$ errors in $n = 10$ days) and the Jeffreys non-informative prior, there is a 95% posterior probability that the true daily error rate λ lies between approximately 2.48 and 4.70. Crucially, the null value $\lambda_0 = 2$ lies *below* this entire interval, providing further confirmation, which is also consistent with the Bayes Factor analysis of Exercise 2, that the data strongly favor a higher error rate than 2 per day.

Comparison with Exercise 2. The Gamma$(2, 1)$ prior gave a posterior Gamma$(37, 11)$ with mean ≈ 3.36 and 95% CI $\approx (2.41, 4.44)$. The Jeffreys prior yields a slightly higher and wider posterior (mean 3.55, CI $\approx (2.48, 4.81)$), because it places no prior mass near $\lambda = 2$ and lets the data speak more freely.

Algorithm convergence. The trace plot shows a stationary, well-mixed chain with no drift. The ergodic mean converges rapidly to the exact posterior mean. The ACF decays to zero within $\sim$10-15 lags and the ESS is well above 1 000. All diagnostics confirm that the MH chain provides a reliable sample from the target posterior.